SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 응용

완전 탐색

Python

학습시작

조건에 맞는 코디를 골라 봅시다!

옷장에 상의 4벌(A, B, C, D)과 하의 2벌(a, b)이 있습니다. 각각 한 벌씩 골라 입으려면 어떤 코디가 가능할까요? 단, A 상의와 b 하의는 어울리지 않기 때문에 함께 입을 수 없습니다.

→ 가능한 모든 조합을 하나씩 만들어 보고, 조건에 맞는 경우만 선택하는 방식을 [ ] 이라고 합니다.

→ 한 가지 선택을 하고, 남은 경우를 이어서 고르는 흐름은 [ ] 구조로 표현할 수 있습니다.

- 조건을 만족하는 정답을 찾기 위해 모든 경우를 만들어보는 흐름을 구현할 수 있습니다.
- 여러 가지 선택지를 순차적으로 이어가며 조합을 완성하는 과정을 설계할 수 있습니다.
- 선택한 값을 기억하고, 되돌아가며 다른 경우를 탐색하는 구조를 직접 구현할 수 있습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

완전 탐색

- 반복과 재귀
- 순열
- 완전탐색
- 연습 문제
- 실습 문제

Objectives

학습목표

- 재귀 호출의 구조와 특성을 이해하고, 이를 활용한 알고리즘을 구현할 수 있다.
- 재귀 호출을 이용해 중첩 반복 구조를 구성하고, 완전 탐색 알고리즘을 설계할 수 있다.
- 순열, 조합, 부분집합 등 조합적 문제를 재귀 호출로 구현할 수 있다.
- 완전 탐색(Brute-force)의 개념을 이해하고, 다양한 조합을 모두 시도하는 문제 해결 전략을 익힌다.
- 주사위 눈 합, 연속 3장의 트럼프 카드, Baby-Jin 문제를 통해 재귀호출을 이용한 완전 탐색 구현 방법에 대해 학습한다.

완전 탐색

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

반복과 재귀

반복(Iteration)과 재귀(Recursion)는 유사한 작업을 수행할 수 있다.

반복은 수행하는 작업이 완료될 때까지 계속 반복

- 루프 (for, while 구조)
- 반복문은 코드를 n 번 반복시킬 수 있음

재귀는 주어진 문제의 해를 구하기 위해 동일하면서 더 작은 문제의 해를 이용하는 방법

- 하나의 큰 문제를 해결할 수 있는(해결하기 쉬운) 더 작은 문제로 쪼개고 결과들을 결합
- 재귀호출은 n 중 반복문과 같은 효과

1, 2 ,3으로 만들 수 있는 두 자릿수 만들기

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

위 문제를 반복문을 이용해서 구현할 수 있을까?

1, 2, 3으로 만들 수 있는 두 자릿수 코드

- 2중 for문을 이용하여 구현

```
for a in range(1, 4):
    for b in range(1, 4):
        print(a, b)
```

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

[도전] 1, 2, 3을 이용한 네 자릿수를 출력하는 코드 작성 (제한시간 : 2분)

```
1 1 1 1
1 1 1 2
1 1 1 3
1 1 2 1
1 1 2 2
   ...
3 3 3 2
3 3 3 3
```

N을 입력받아 1, 2, 3을 사용하는 N 자릿수 만들기

- 만약 N = 2 면 1 1 ~ 3 3 까지 출력
- 만약 N = 3 이면 1 1 1 ~ 3 3 3 까지 출력
- 만약 N = 4 면 1 1 1 1 ~ 3 3 3 3 까지 출력
- …
- 만약 N = 9 면 1 1 1 1 1 1 1 1 1 ~ 9 9 9 9 9 9 9 9 9 까지 출력

위 문제는 반복문을 이용해서 구현할 수 있을까?

구현할 수 있다면 몇 중 for문을 작성해야 할까?

재귀호출로 N 중 for문 구현

- N 입력 후 1 1 1… ~ 3 3 3 … 출력하는 문제는 for문으로 구현이 어려움
- 아래처럼 재귀호출로 구현할 수 있다.

```python
path = []
N = 3

def run(lev):
    if lev == N:
        print(path)
        return
    for i in range(1, 4):
        path.append(i)
        run(lev + 1)
        path.pop()

#N = int(input())
run(0)
```

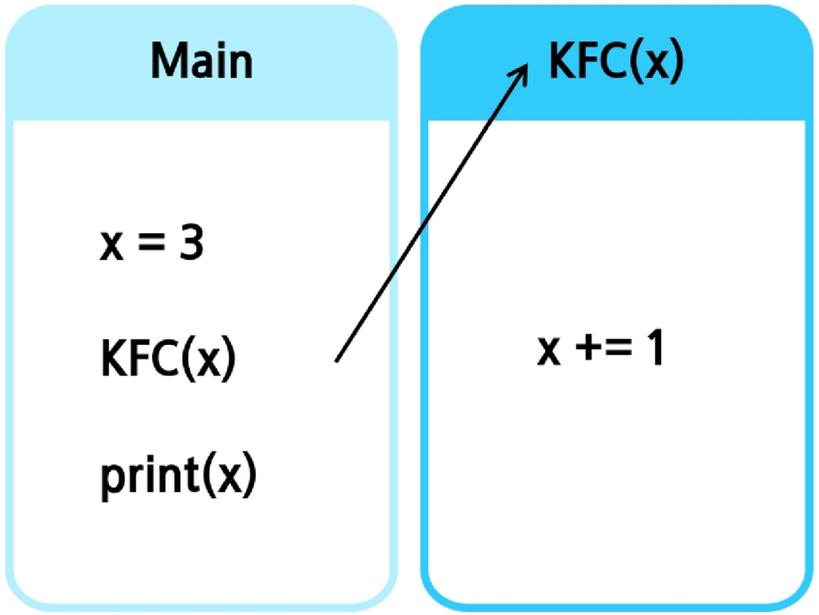
◀ 재귀 호출로 구현한 코드

```
[1, 1, 1]
[1, 1, 2]
[1, 1, 3]
[1, 2, 1]
[1, 2, 2]
[1, 2, 3]
[1, 3, 1]
[1, 3, 2]
[1, 3, 3]
[2, 1, 1]
[2, 1, 2]
…
[3, 3, 3]
```

재귀를 연습하기 전, 알아야 할 함수의 특징 1

- KFC 함수 호출할 때, int 타입 객체를 전달하면 값만 복사됨
- 아래 예시에서, main 함수의 x와, KFC함수의 x는 서로 다른 객체

 - 마치 한국 사는 KIM과, 미국 사는 KIM이 이름만 같고, 서로 다른 사람인 것과 같습니다.



출력 결과는 4가 아니라 3

Main함수의 변수 x가
KFC함수의 변수 x로 값만 복사된 것

<그림1_알고리즘 응용_완전탐색_재귀를 연습하기 전, 알아야 할 함수의 특징 1>

재귀를 연습하기 전, 알아야 할 함수의 특징 1

- KFC 함수 호출할 때, int 타입 객체를 전달하면 값만 복사됨

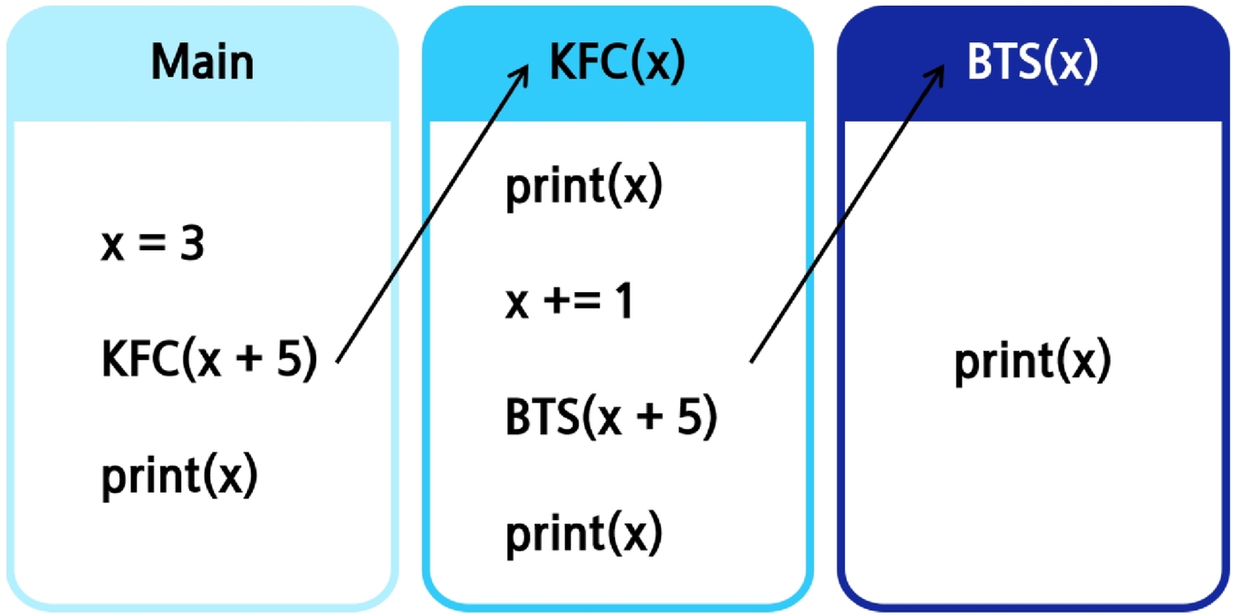
- 아래 코드의 실행결과?

```
def KFC(x):
    print(x)
    x += 1
    print(x)

x = 3
KFC(x + 1)
print(x)
```

재귀를 연습하기 전, 알아야 할 함수의 특징 2

- BTS 함수가 끝나면, Main으로 되돌아 오는 것이 아니라,
  해당 함수를 호출했던 곳으로 돌아옴

- 아래 코드의 실행 결과는?



<그림2_알고리즘 응용_완전탐색_재귀를 연습하기 전, 알아야 할 함수의 특징 2>

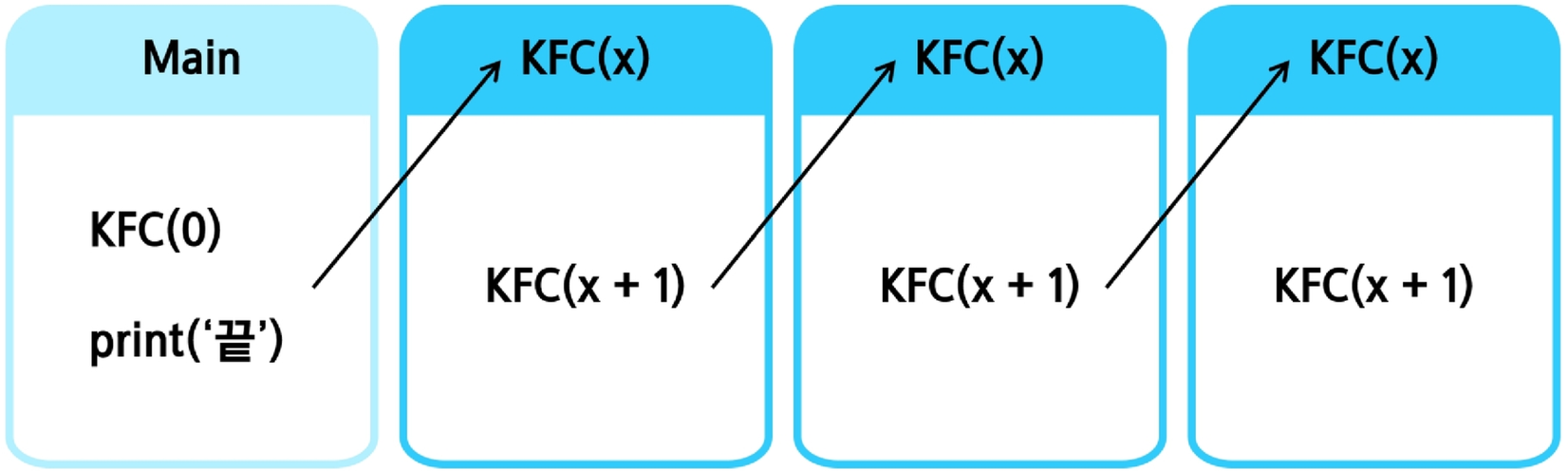
재귀를 연습하기 전, 알아야 할 함수의 특징 2

- 함수가 끝나면, 해당 함수를 호출했던 곳으로 돌아옴
- 다음 코드의 실행 결과는?

```
def BBQ(x):
    x += 10
    print(x)

def KFC(x):
    print(x)
    x += 3
    BBQ(x + 2)
    print(x)

x = 3
KFC(x + 1)
print(x)
```

[도전] 무한 재귀호출 (제한시간 : 2분)

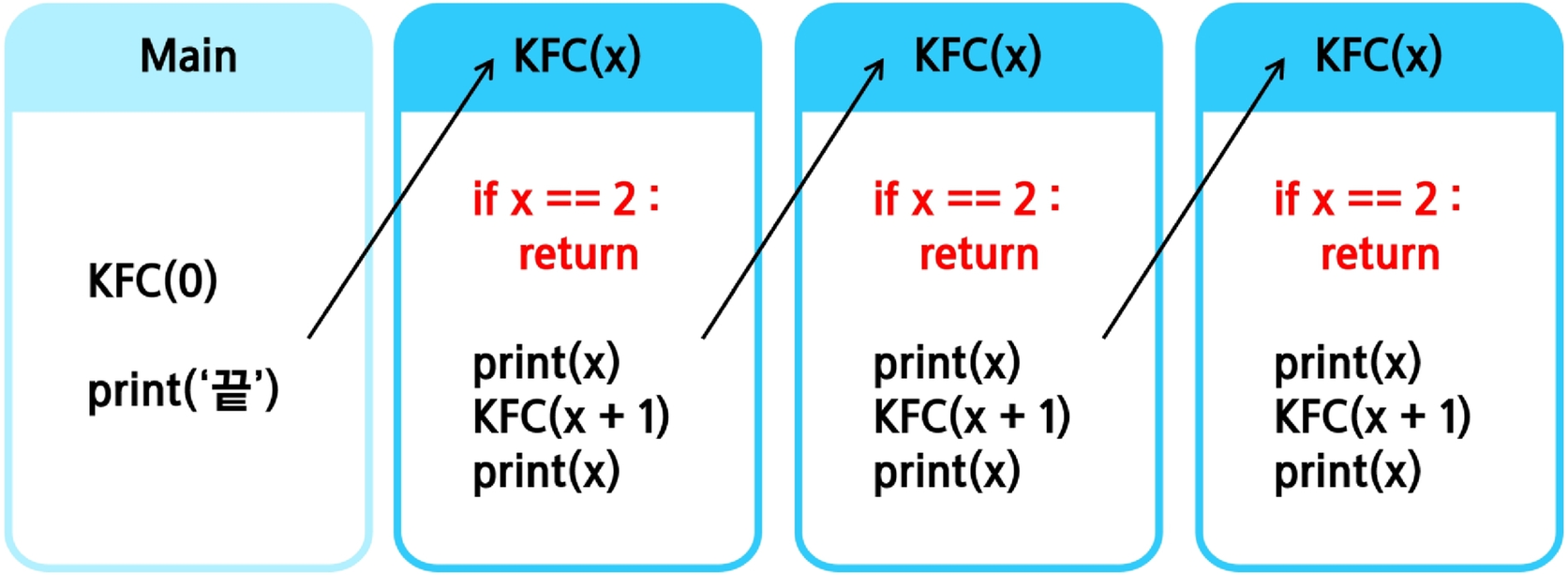
- 아래와 그림과 같은 코드를 직접 구현
  - 첫 번째 KFC 함수의 x 값은 0이 됩니다.
  - 두 번째 KFC 함수의 x 값은 1이 됩니다.
  - 세 번째 KFC 함수의 x 값은 2가 됩니다.
- 무한 재귀 호출이 발생하면 어떻게 되는지, 직접 코드를 작성하고 실행



<그림3_알고리즘 응용_완전탐색_무한 재귀호출 >

재귀호출 공부의 시작은, 무한 재귀호출을 막는 것부터 시작

- 출력 결과는 0 1 1 0
- 아래 if문을 “기저조건 (base case)” 이라고 함



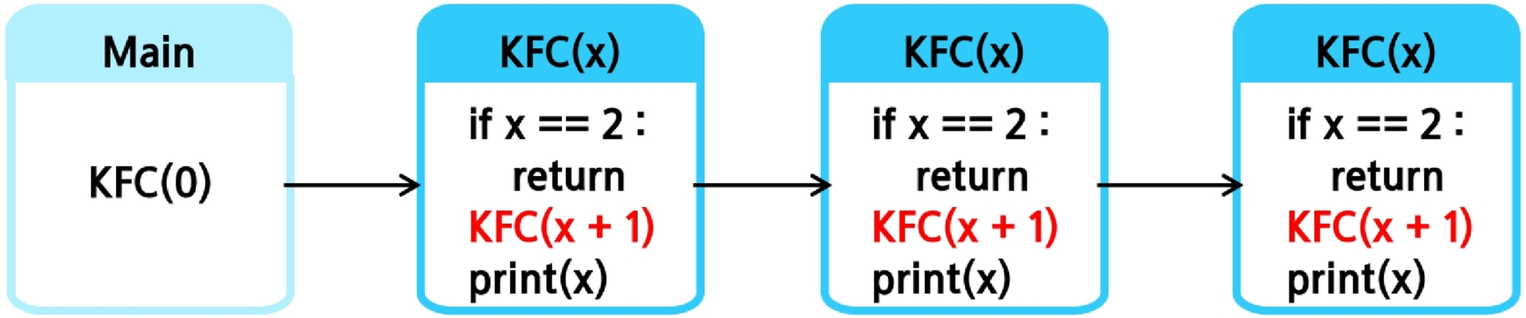
<그림4_알고리즘 응용_완전탐색_기저조건>

[도전] 0 1 2 3 4 5 5 4 3 2 1 0 을 재귀호출을 이용하여 구현 (제한시간 : 3분)

몇 번 Depth에서 return 을 해야 하고,
print 문을 어느 위치에 배치할지 고민해봅시다.

재귀호출 코드가 1개

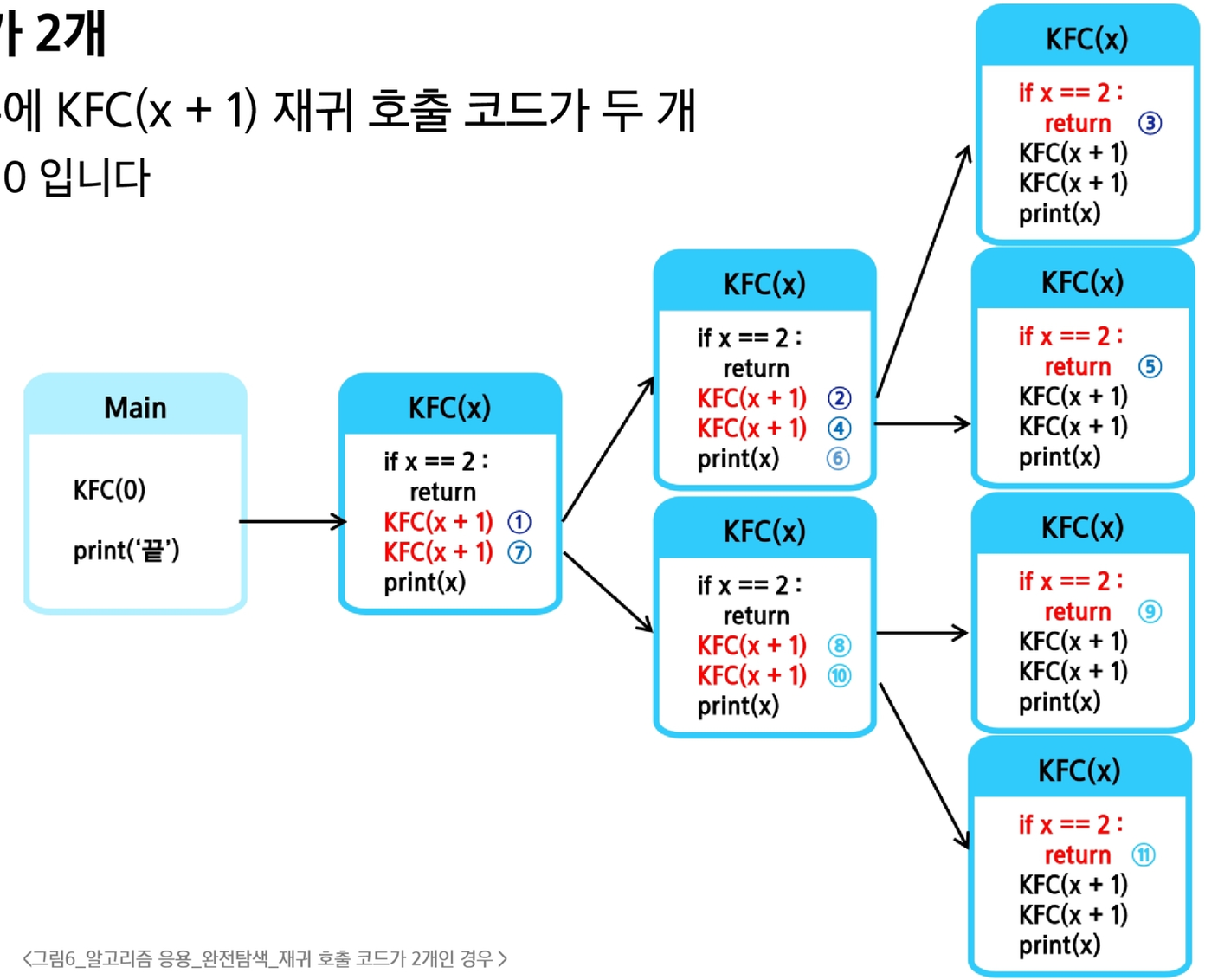
- KFC 함수 내부에 KFC(x + 1) 재귀 호출 코드가 하나인 경우 예시 그림
  - 출력 결과는 1 0 입니다.



<그림5_알고리즘 응용_완전탐색_재귀 호출 코드가 1개인 경우>

재귀호출 코드가 2개

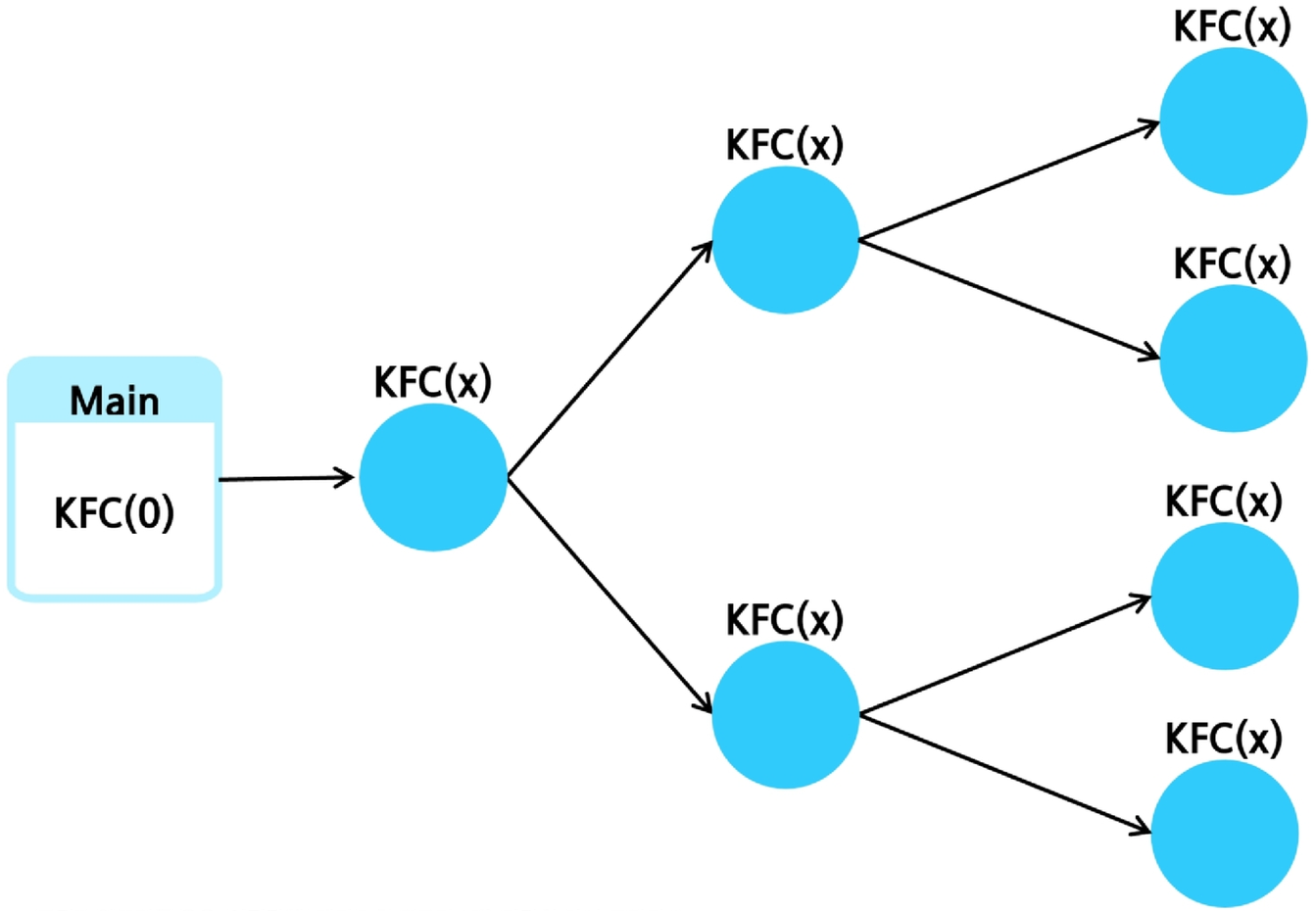
- KFC 함수 내부에 KFC(x + 1) 재귀 호출 코드가 두 개
  - 출력 결과는 1 1 0 입니다



<그림6_알고리즘 응용_완전탐색_재귀 호출 코드가 2개인 경우 >

재귀호출 코드가 2개, 간략화 하여 그리기 1

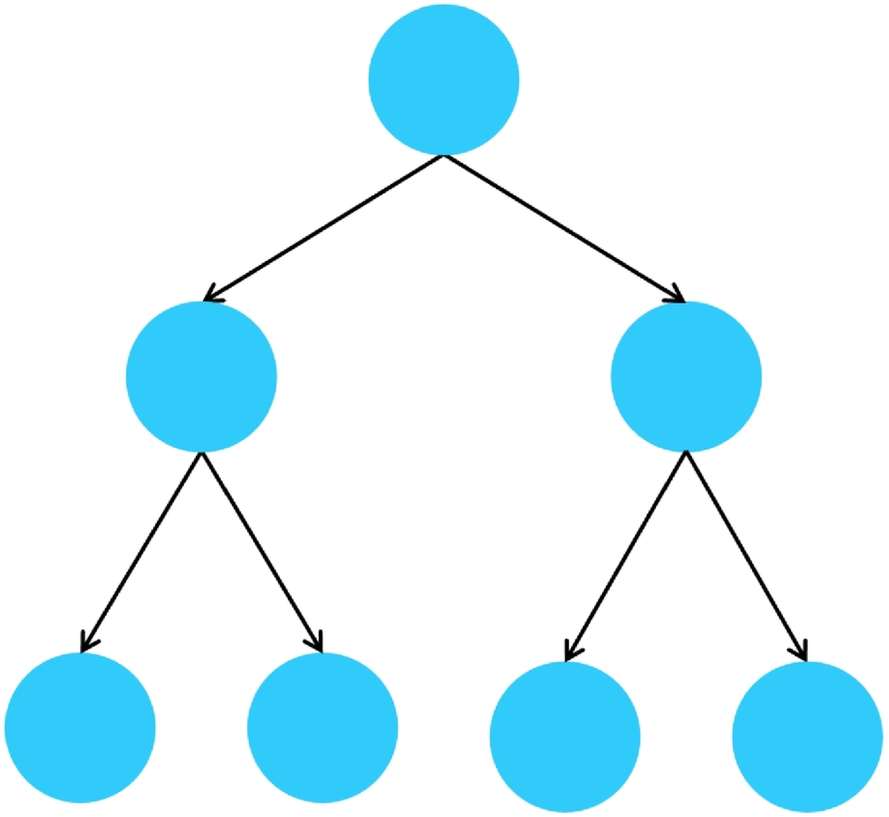
- KFC 함수 내부에 KFC(x + 1) 재귀 호출 코드가 두 개



<그림7_알고리즘 응용_완전탐색_재귀호출 코드가 2개, 간략화 하여 그리기 1>

재귀호출 코드가 2개, 간략화 하여 그리기 2

- 아래 그림만 보고, 우측 코드를 구현할 수 있어야 함

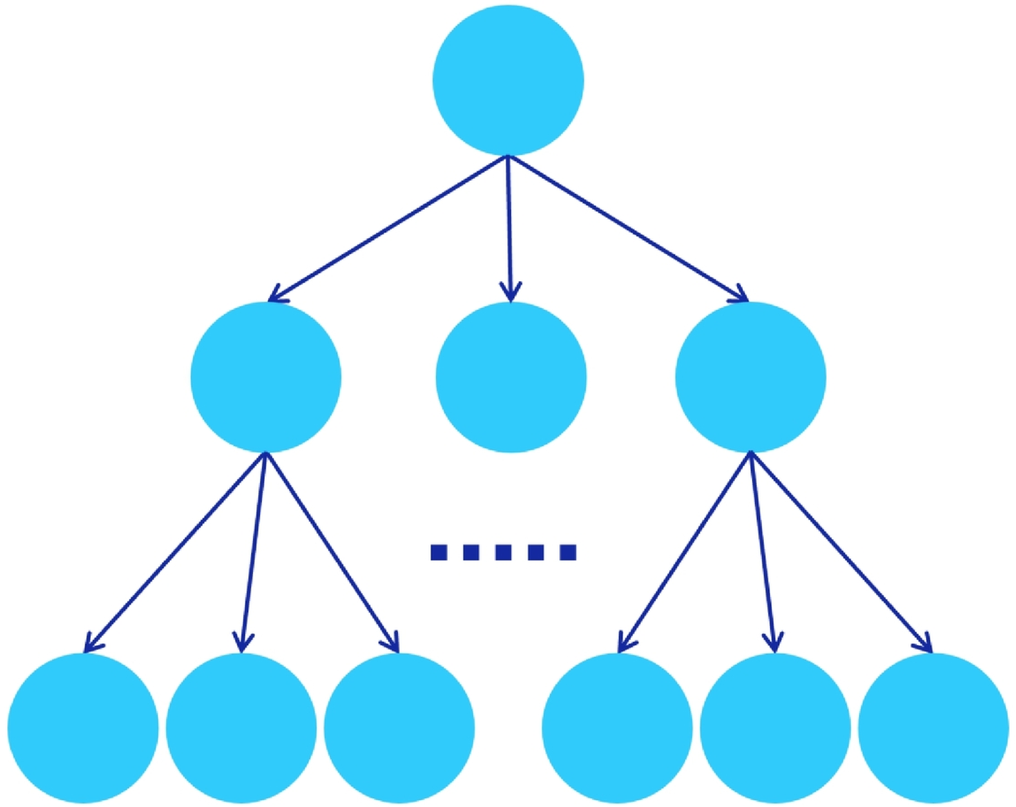


<그림8_알고리즘 응용_완전탐색_재귀호출 코드가 2개, 간략화 하여 그리기 2>

```
def KFC(x):
    if x == 2:
        return

    KFC(x + 1)
    KFC(x + 1)
    print(x)

KFC(0)
```

재귀호출 코드가 3개

- KFC 함수 내부에 KFC(x + 1) 재귀 호출 코드가 세 개

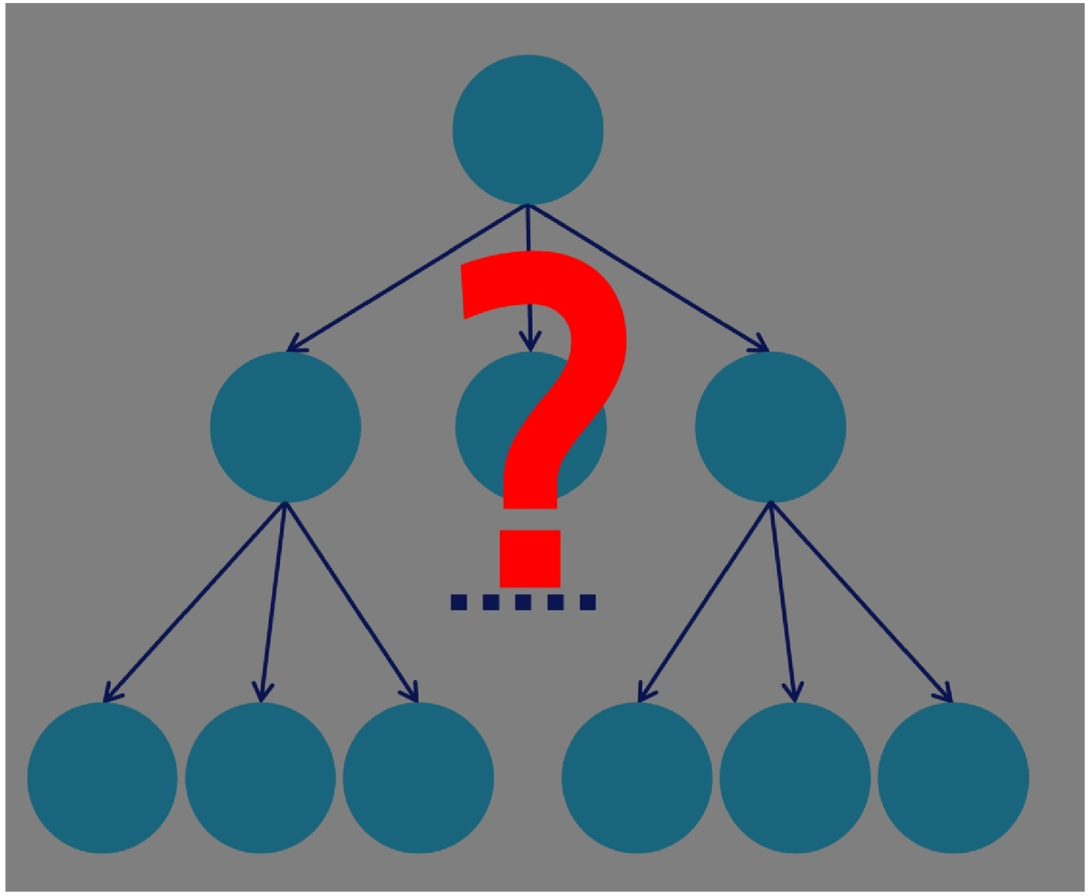


<그림9_알고리즘 응용_완전탐색_재귀 호출 코드가 3개인 경우>

```
def KFC(x):
    if x == 2:
        return


    KFC(x + 1)
    KFC(x + 1)
    KFC(x + 1)
    print(x)


KFC(0)
```

[도전] Depth 3, 재귀호출 개수 4개인 경우 그림 그려보기 (제한시간 : 1분)

- 그림판에 직접 트리형태로 그려보기



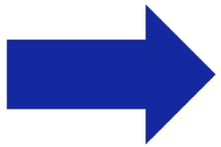
<그림10_알고리즘 응용_완전탐색_재귀호출 개수 4개인 경우>

```
def KFC(x):
    if x == 3:
        return

    KFC(x + 1)
    KFC(x + 1)
    KFC(x + 1)
    KFC(x + 1)
    print(x)

KFC(0)
```

깊이를 Level이라고 부름

- Level은 3, Branch (나뭇가지)는 4인 형태인 코드를 for문으로도 표현 가능

```
def KFC(x):
    if x == 3:
        return

    KFC(x + 1)
    KFC(x + 1)
    KFC(x + 1)
    KFC(x + 1)

KFC(0)
```



```
def KFC(x):
    if x == 3:
        return

    for i in range(1, 4):
        KFC(x + 1)

print(0)
```

재귀호출을 이용한 코딩 기법 학습

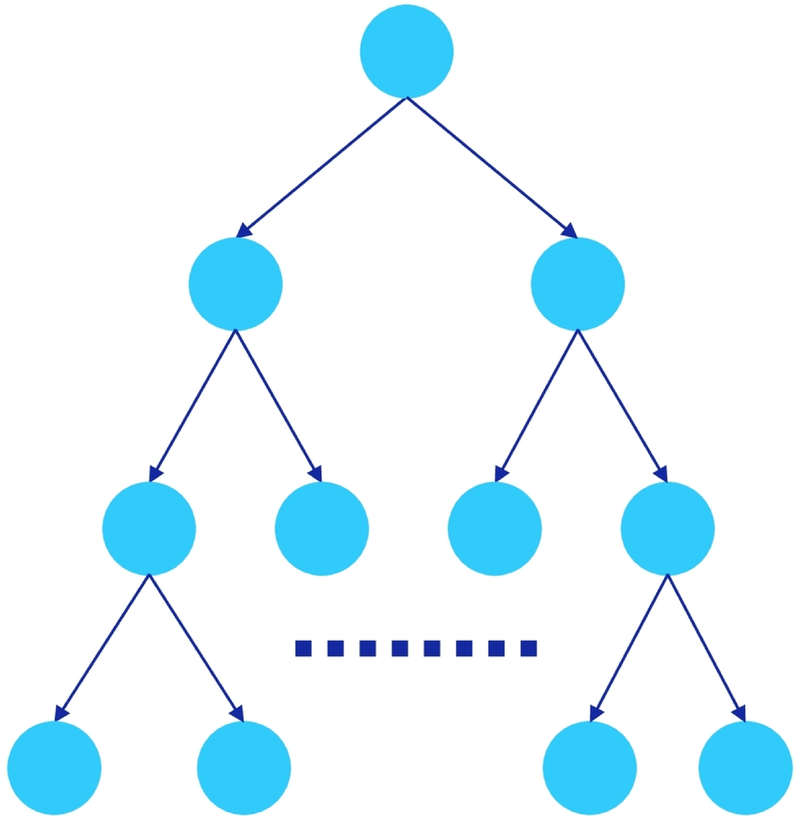
- 반드시 아래 코드의 원리를 이해해야 함

- 이제 순열을 학습할 준비를 마쳤습니다.

```
def KFC(x):
    if x == 3:
        return

    for i in range(1, 4):
        KFC(x + 1)

KFC(0)
```

[도전] 트리 형태를 보고, 직접 재귀호출 코드 구현하기

Branch ? Level ? 을 먼저 파악한다. 이후 코드를 직접 구현한다.



<그림11_알고리즘 응용_완전탐색_트리형태 보고 재귀호출 코드 구현하기>

순열

순열이란?

- 서로 다른 N개에서, R개를 중복 없이, 순서를 고려하여 나열하는 것.

- 예시) 3개에서 2개를 뽑아 만드는 순열

 - [0] [1] [2] 로 구성된 3장의 카드가 다량으로 존재합니다.

 - 이 중에 2장을 뽑아, 순열을 나열합니다.

(순열은 중복을 취급하지 않는다.)

<그림12_알고리즘 응용_완전탐색_순열 카드>

[0 1]
[0 2]
[1 0]
[1 2]
[2 0]
[2 1]

순열

중복순열이란?

- 서로 다른 N개에서, R개를 중복을 허용하고, 순서를 고려하여 나열하는 것.

- 예시)

- [0] [1] [2] 로 구성된 3장의 카드가 다량으로 존재합니다.
- 이 중에 2장을 뽑아, 중복순열을 나열합니다.

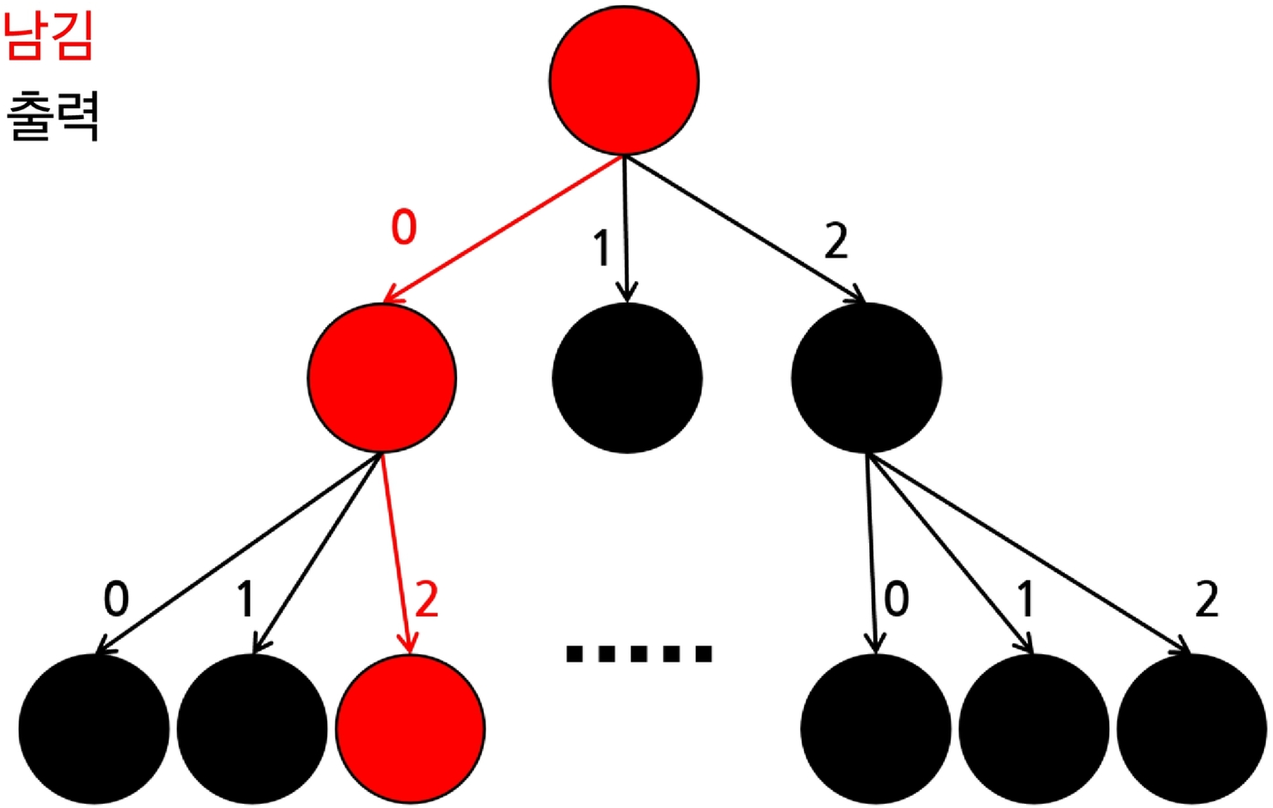
[0 0]
[0 1]
[0 2]
[1 0]
[1 1]
[1 2]
[2 0]
[2 1]
[2 2]

중복순열 구현 원리

1. 재귀호출을 할 때 마다, 이동 경로를 흔적으로 남김
2. 가장 마지막 레벨에 도착했을 때, 이동 경로를 출력

<그림13_알고리즘 응용_완전탐색_중복 순열 구현 원리>

'헨젤과 그레텔' 동화의 빵가루처럼,
어디에서 왔었는지 기록해둔다.

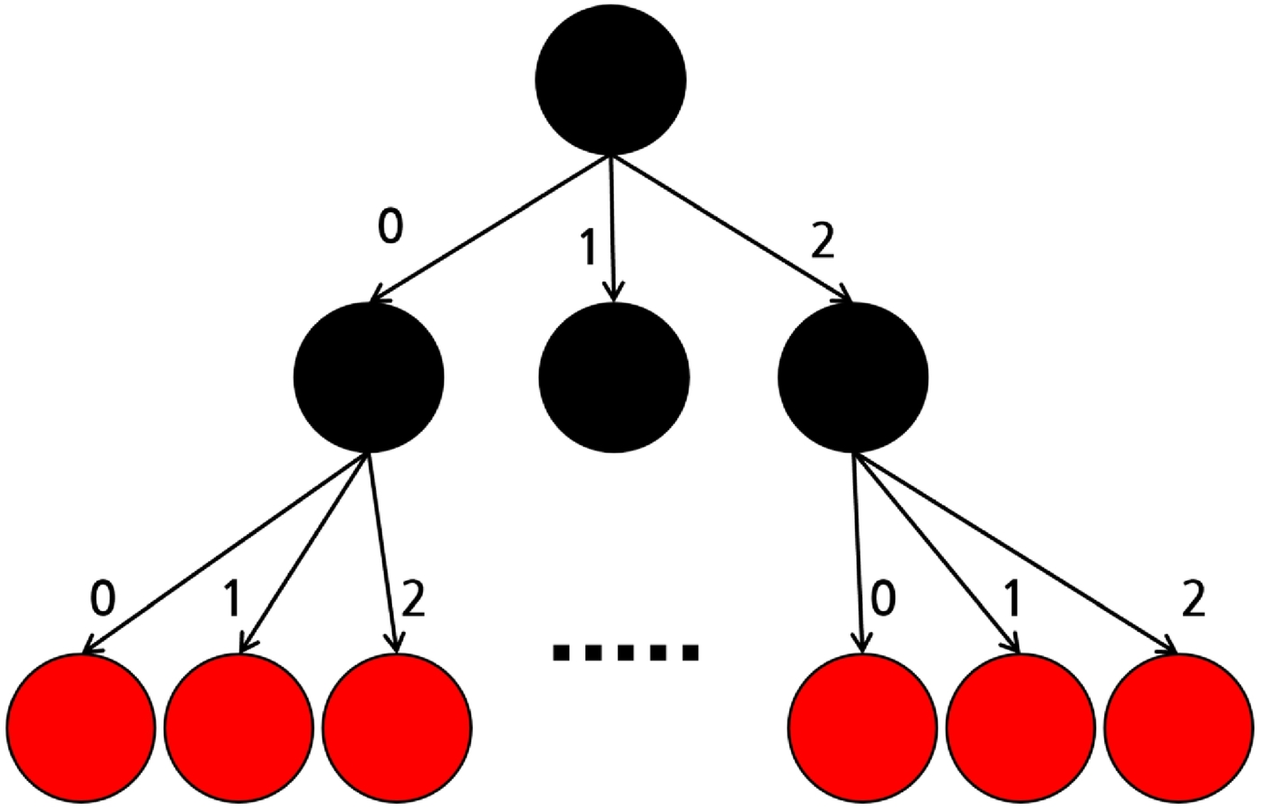


이곳에 도착했다면,
흔적을 [0 2] 으로 남긴다.

<그림14_알고리즘 응용_완전탐색_중복 순열 구현 1>

중복순열 구현 원리

1. 재귀호출을 할 때 마다, 이동 경로를 흔적으로 남김
2. 가장 마지막 레벨에 도착했을 때, 이동 경로를 출력



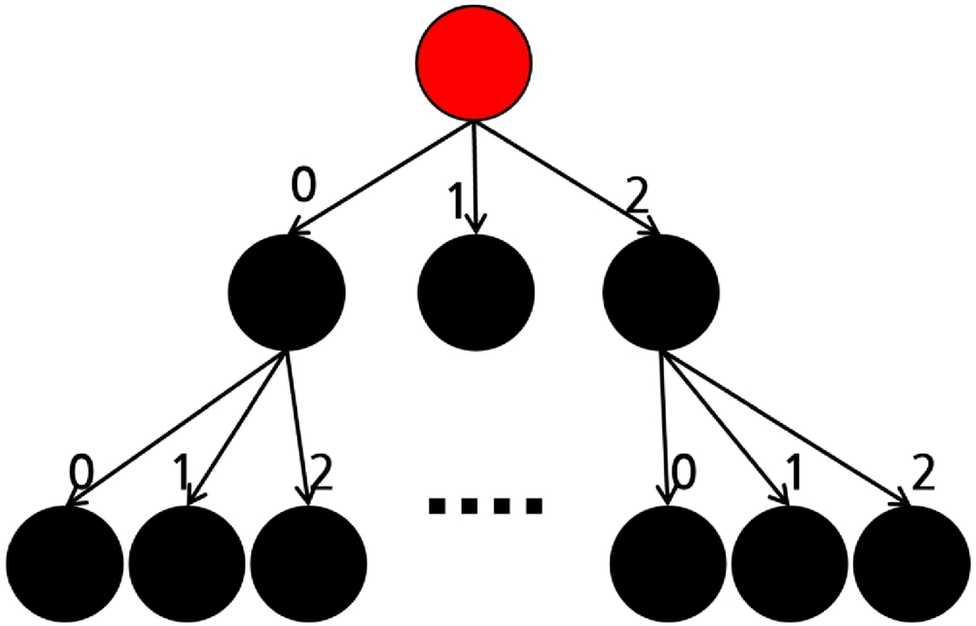
<그림15_알고리즘 응용_완전탐색_중복 순열 구현 2>

마지막 레벨에 도착할 때 마다
출력하면 출력 결과는 다음과 같다.

```
[0 0]
[0 1]
[0 2]
[1 0]
[1 1]
[1 2]
[2 0]
[2 1]
[2 2]
```

순열 (Permutation)

- 먼저 path라는 전역 리스트를 준비한다.
- 그리고 Level 2, Branch 3으로 동작하는 재귀 코드를 구현



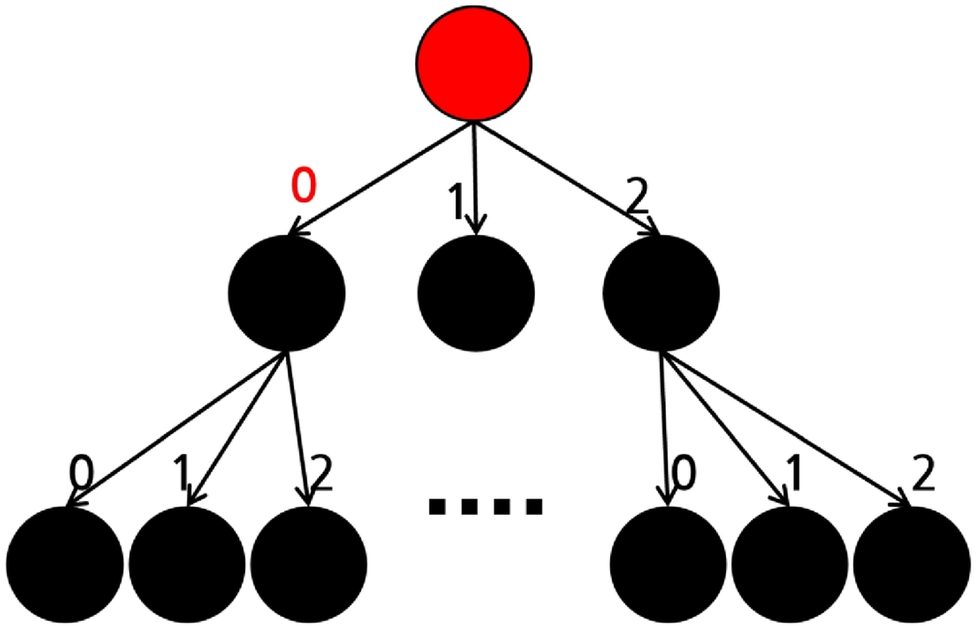
재귀호출 시작 전에는
path = [ ] 이 된다.

<그림16_알고리즘 응용_완전탐색_순열 구현 1>

```
path = []

def KFC(x):
    if x == 2:
        return

    for i in range(3):
        KFC(x + 1)

KFC(0)
```

순열 (Permutation)

- 재귀호출을 하기 직전에 이동할 곳의 위치를 path 리스트에 기록



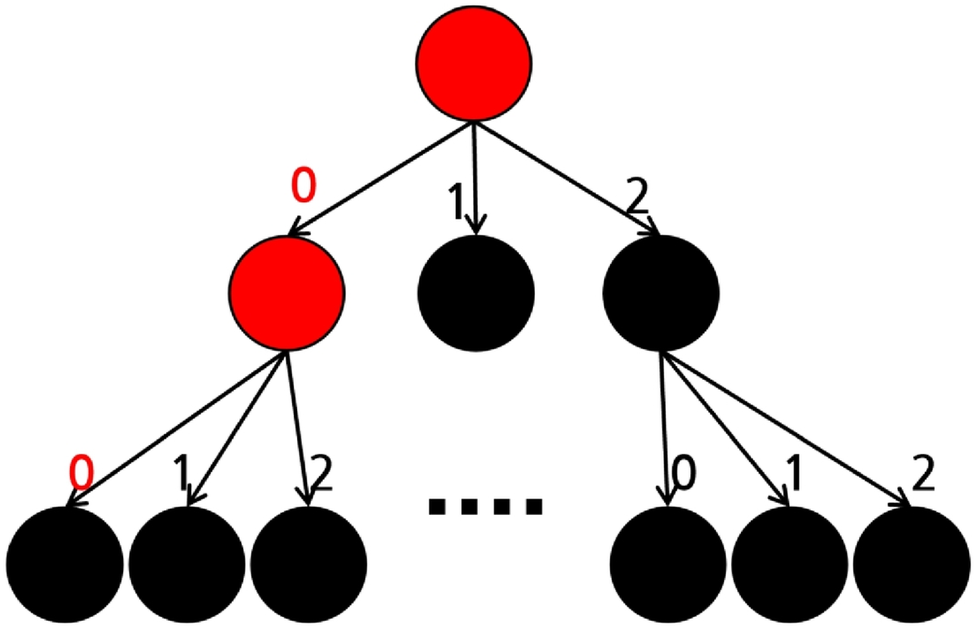
재귀호출 직전 path는
path = [0]이 되고
재귀호출을 한다.

```
path = []

def KFC(x):
    if x == 2:
        return

    for i in range(3):
        path.append(i)
        KFC(x + 1)

KFC(0)
```

append 코드 추가!

순열 (Permutation)

- 재귀호출 됨. 그리고 코드가 계속 진행되어, path.append(i)를 수행



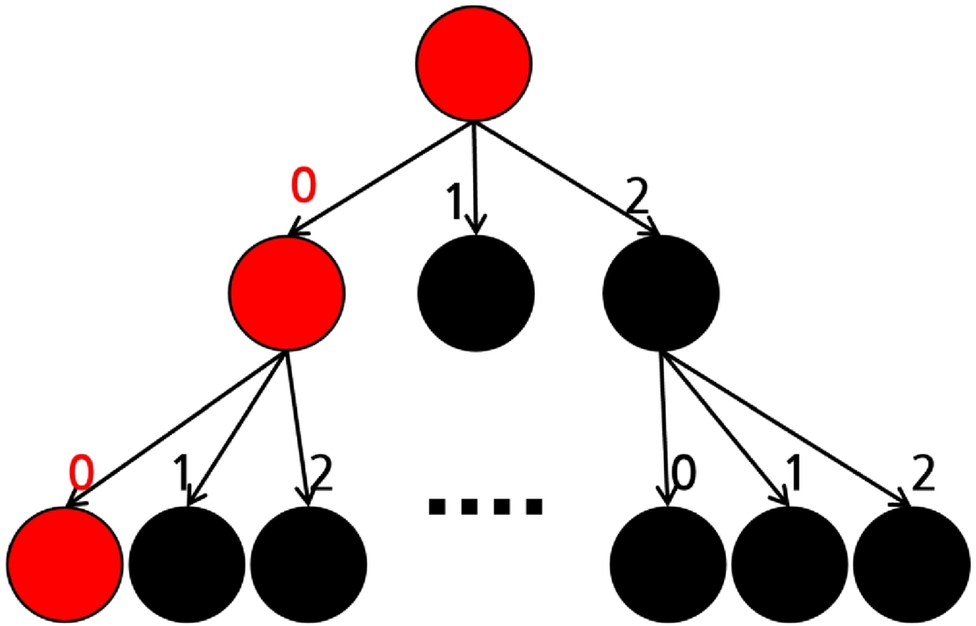
재귀호출 후
append를 한번 더 수행하여
path = [0, 0]이 된다.

```python
path = []

def KFC(x):
    if x == 2:
        return

    for i in range(3):
        path.append(i)
        KFC(x + 1)

KFC(0)
```

<그림18_알고리즘 응용_완전탐색_순열 구현 3>

순열 (Permutation)

- 두 번 재귀호출 되었고, 이제 바닥에 도착했으니 출력하는 코드를 수행



path = [0, 0] 이므로
출력결과는 [0, 0] 이다.

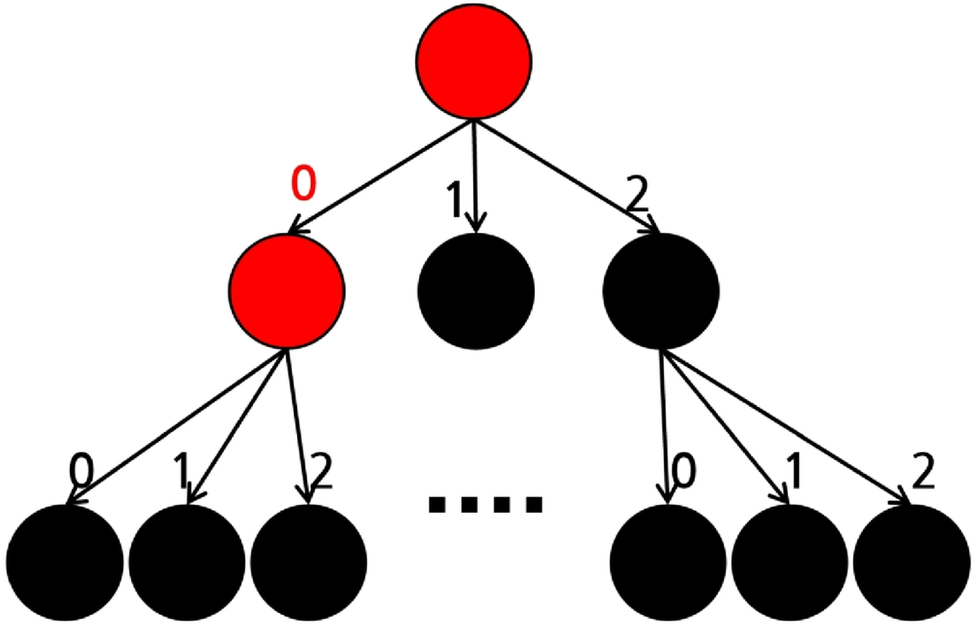
<그림19_알고리즘 응용_완전탐색_순열 구현 4>

```
path = []

def KFC(x):
    if x == 2:
        print(path)
        return

    for i in range(3):
        path.append(i)
        KFC(x + 1)

KFC(0)
```

출력 코드 추가!

순열 (Permutation)

- 함수가 리턴 되고, 함수가 즉시 종료됨
- 이후 path에 적은 마지막 기록이 삭제 되어야 함



되돌아 온 후, 기록을 삭제한다.
Path = [0, 0] 에서
Path = [0] 이 된다.

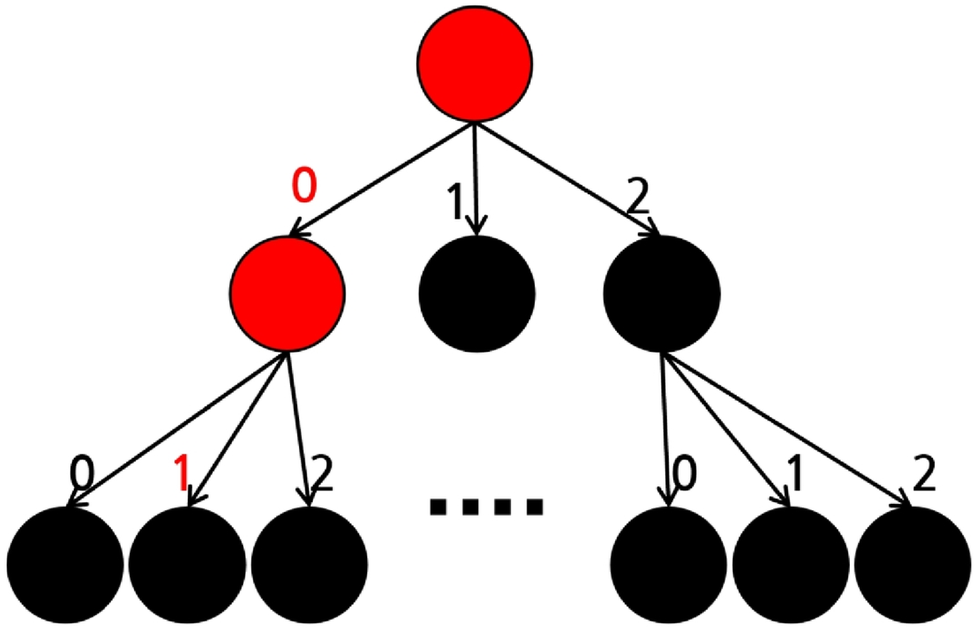
<그림20_알고리즘 응용_완전탐색_순열 구현 5>

```
path = []

def KFC(x):
    if x == 2:
        print(path)
        return

    for i in range(3):
        path.append(i)
        KFC(x + 1)
        path.pop()

KFC(0)
```

기록 삭제 코드 추가!

순열 (Permutation)

- 이어서 for문이 진행되고, 변수 i 값은 1 이 됨
- path 배열 마지막에 1 이 삽입됨



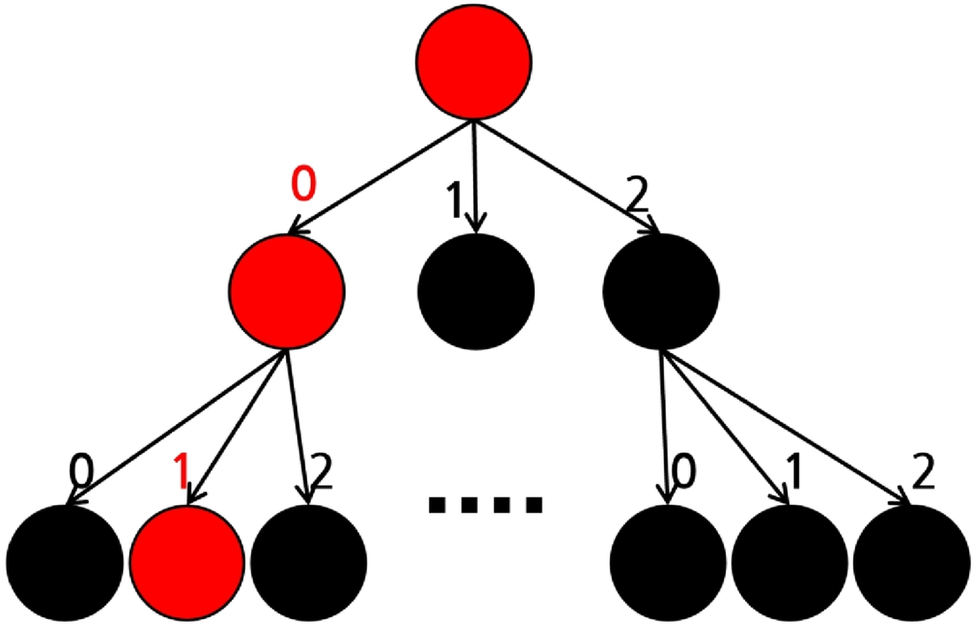
다시 1을 path에 집어 넣는다.
path = [0, 1] 이 된다.

<그림21_알고리즘 응용_완전탐색_순열 구현 6>

```python
path = []

def KFC(x):
    if x == 2:
        print(path)
        return

    for i in range(3):
        path.append(i)
        KFC(x + 1)
        path.pop()

KFC(0)
```

순열 (Permutation)

- 재귀호출이 된 후, path 리스트를 출력



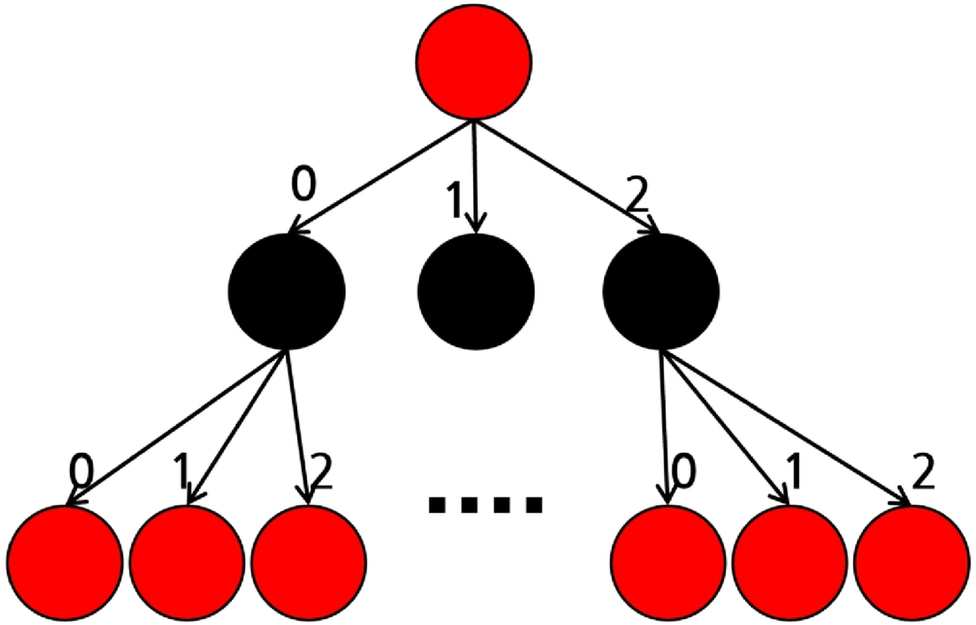
path 를 출력한다.
출력 결과는 [0, 1] 이다.

<그림22_알고리즘 응용_완전탐색_순열 구현 7>

```
path = []

def KFC(x):
    if x == 2:
        print(path)
        return

    for i in range(3):
        path.append(i)
        KFC(x + 1)
        path.pop()


KFC(0)
```

순열 (Permutation)

- 중복 순열 소스코드 완성 → [0, 0] ~ [2, 2] 까지 출력하는 소스코드



마지막 Level에 도착할 때 마다
출력하면 된다.

<그림23_알고리즘 응용_완전탐색_순열 구현 8>

```
path = []

def KFC(x):
    if x == 2:
        print(path)
        return

    for i in range(3):
        path.append(i)
        KFC(x + 1)
        path.pop()


KFC(0)
```

[도전] 1부터 6까지 사용하는 중복순열([1, 1, 1] ~ [6, 6, 6])을 출력하는 코드를 재귀호출로 구현 (처음부터 구현하여, 제한시간 5분)

출력 결과

```
1 1 1
1 1 2
1 1 3
1 1 4
1 1 5
1 1 6
1 2 1
...
6 6 5
6 6 6
```

도전 과제를 성공적으로 해결했다면,
[1 1 1 1 1] ~ [4 4 4 4 4] 중복순열 코드도 구현해보세요.

중복을 취급하지 않는 “순열” 구현 방법

1. 중복순열 코드를 작성
2. 중복을 제거하는 코드를 추가하면 순열 코드가 됨

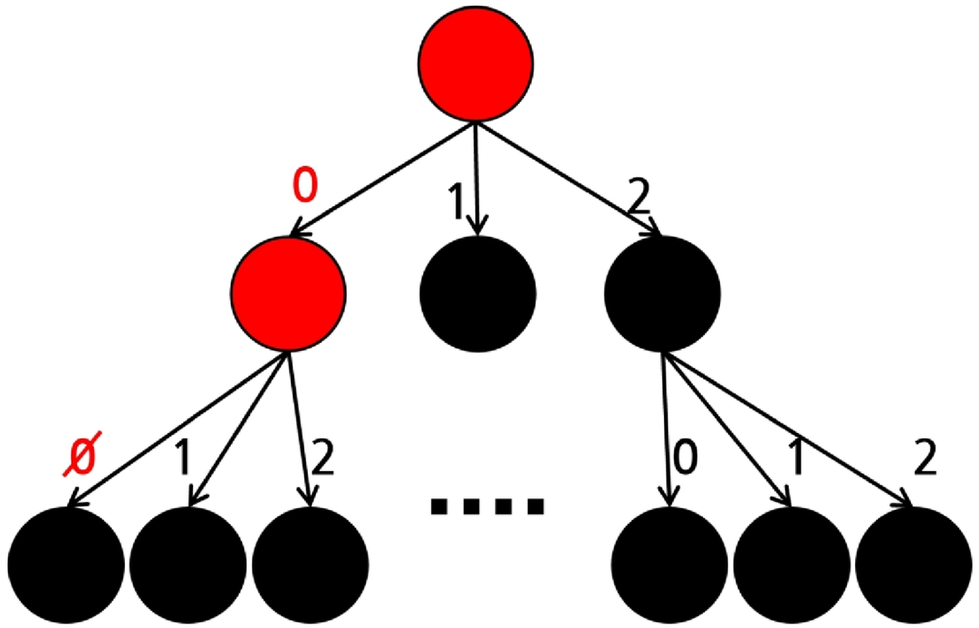
중복을 제거하는 원리

- 전역 리스트를 사용해서 이미 선택했던 숫자인지 아닌지 구분

- 이를 used 배열 또는 visited 배열 이라고 부름
  - DFS, BFS에 사용되는 것과 같습니다.

중복을 제거한 순열 생성 예시

- 0을 선택하고 재귀호출 한 후에는, 또 다시 0을 선택하지 못하도록 막아야 함
- 재귀 호출을 하기 직전, 이미 선택했던 숫자인지 아닌지 검사하는 코드가 필요



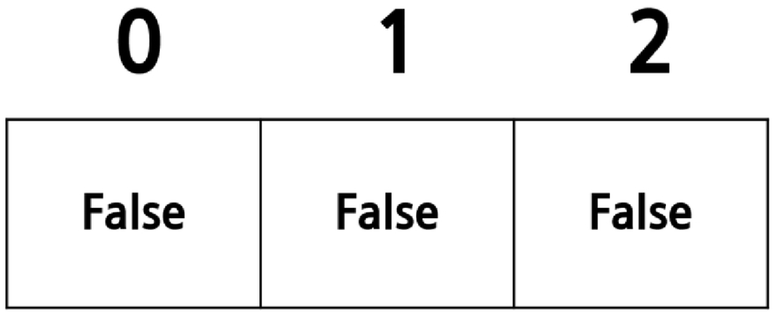
<그림24_알고리즘 응용_완전탐색_중복을 제거한 순열 생성 예시>

중복을 제거한 순열 생성 예시

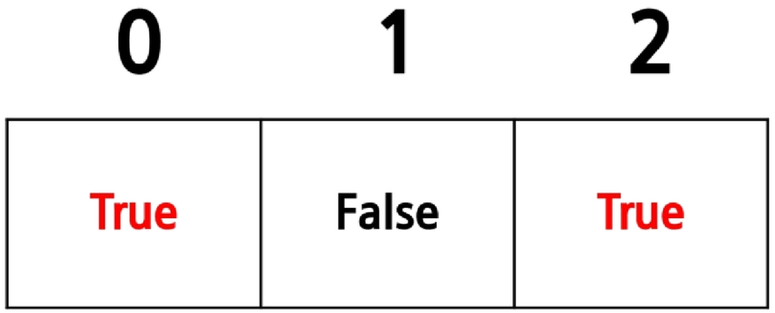
- 이미 사용한 숫자인지 아닌지 구분하는 List 준비하기

 - 전역으로 used 라는 리스트를 준비합니다.

```
used = [False, False, False]
# 아래와 같이 리스트 내포 문법을 사용해도 된다.
# used = [False for _ in range(3)]
```



0, 1, 2번을 아직 선택 안했을 때



0, 2번을 사용했었고,
1번은 아직 사용 안했을 때

<그림25_알고리즘 응용_완전탐색_사용한 숫자 구분 List>

중복을 제거한 순열 생성 예시

- 이미 사용을 한 숫자인지 아닌지 검사해주는 소스코드 추가

- 만약 이미 사용한 숫자일 경우, 재귀호출을 생략하는 코드를 추가

```
path = []
used = [False, False, False]

def KFC(x):
    if x == 2:
        print
        return

    for i in range(3):
        if used[i] == True:
            continue
        used[i] = True
        path.append(i)
        KFC(x + 1)
        path.pop()
        used[i] = False

KFC(0)
```

중복을 제거한 순열 생성 예시

- 처음 사용하는 숫자라면

used에 기록함

- 모든 처리가 끝나고 돌아왔다면,

used에서 기록을 지움

```
path = []
used = [False, False, False]

def KFC(x):
    if x == 2:
        print
        return

    for i in range(3):
        if used[i] == True:
            continue
        used[i] = True
        path.append(i)
        KFC(x + 1)
        path.pop()
        used[i] = False

KFC(0)
```

[도전] 중복 순열과 순열 구현하기 (제한시간 : 5분)

- N개의 주사위를 던져 나올 수 있는 모든 중복 순열(Type 1)과 순열(Type 2)를 출력

- 입력)

 - N은 2이고, Type은 1이므로 중복 순열을 출력합니다

```
2 1 #N, Type
```

출력)

```
1 1
1 2
1 3
1 4
1 5
1 6
 …
6 5
6 6
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

완전탐색

완전탐색이란?

완전탐색
(= Brute-Force, 부르트 포스 알고리즘)

모든 가능한 경우를 시도해서 정답을 찾아내는 알고리즘

완전탐색 예시

- 자전거 열쇠 비밀번호 맞추기

- 만약 1111 ~ 9999 네 자리 숫자를 맞춰야 한다면?
  솔루션 → 4중 for문으로 모두 시도해봅니다.

- 만약 1 ~ 9까지 이뤄진 N자리의 숫자를 맞춰야 한다면?
  솔루션 → 순열 코드(재귀호출)로 구현하여, 모두 시도해봅니다.

완전탐색 문제 1. 주사위 눈의 합

- 3개의 주사위를 던져 나올 수 있는 중복 순열에 대해, 합이 10 이하가 나오는 경우는 총 몇 가지 인가?

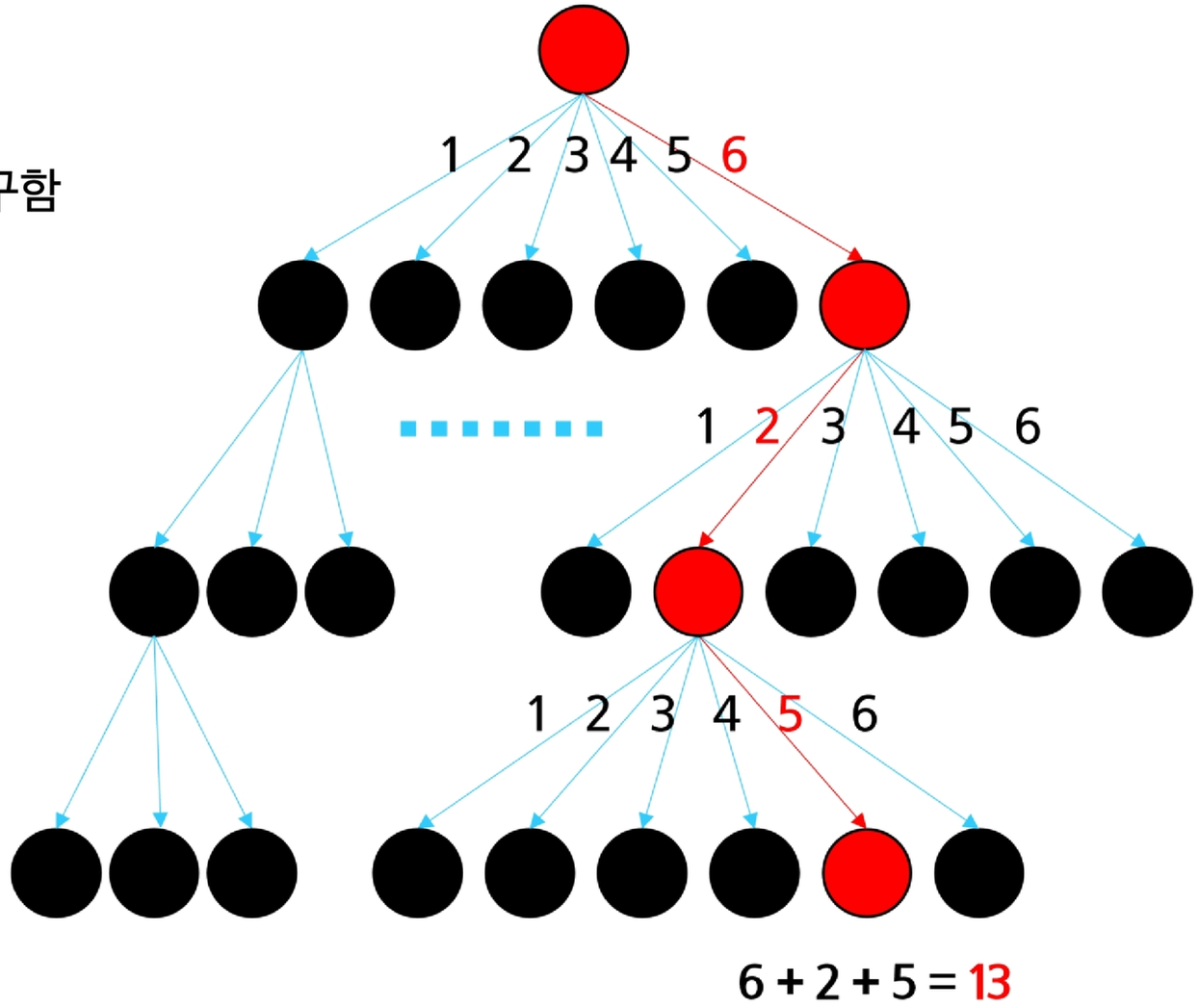
<그림26_알고리즘 응용_완전탐색_3개의 주사위 눈의 합>

가능한 모든 케이스를 탐색한다.

→ 완전탐색 알고리즘 이라고 합니다.

완전탐색 문제 1. 주사위 눈의 합

- 먼저 합을 출력하는 코드를 작성
- 재귀호출을 할 때마다 선택한 값의 누적 합을 구함



<그림27_알고리즘 응용_완전탐색_주사위 눈의 합 누적합>

완전탐색 문제 1. 주사위 눈의 합

- 파라미터에 sum을 추가하여 구현
  - sum : 지금까지 구한 합
  - i : 선택한 주사위 눈금
- 재귀호출 할 때 sum + i 값을 전달

```python
path = []

def kfc(x, sum):
    if x == 3:
        print(f'{path} = {sum}')
        return

    for i in range(1, 7):
        path.append(i)
        kfc(x + 1, sum + i)
        path.pop()

kfc(x: 0, sum: 0)
```

```
[1, 1, 1] = 3
[1, 1, 2] = 4
[1, 1, 3] = 5
[1, 1, 4] = 6
[1, 1, 5] = 7
[1, 1, 6] = 8
[1, 2, 1] = 4
[1, 2, 2] = 5
[1, 2, 3] = 6
[1, 2, 4] = 7
[1, 2, 5] = 8
[1, 2, 6] = 9
⋮
[6, 4, 6] = 16
[6, 5, 1] = 12
[6, 5, 2] = 13
[6, 5, 3] = 14
[6, 5, 4] = 15
[6, 5, 5] = 16
[6, 5, 6] = 17
[6, 6, 1] = 13
[6, 6, 2] = 14
[6, 6, 3] = 15
[6, 6, 4] = 16
[6, 6, 5] = 17
[6, 6, 6] = 18
```

<그림28_알고리즘 응용_완전탐색_주사위 눈의 합 누적합 출력값>

완전탐색 문제 1. 주사위 눈의 합

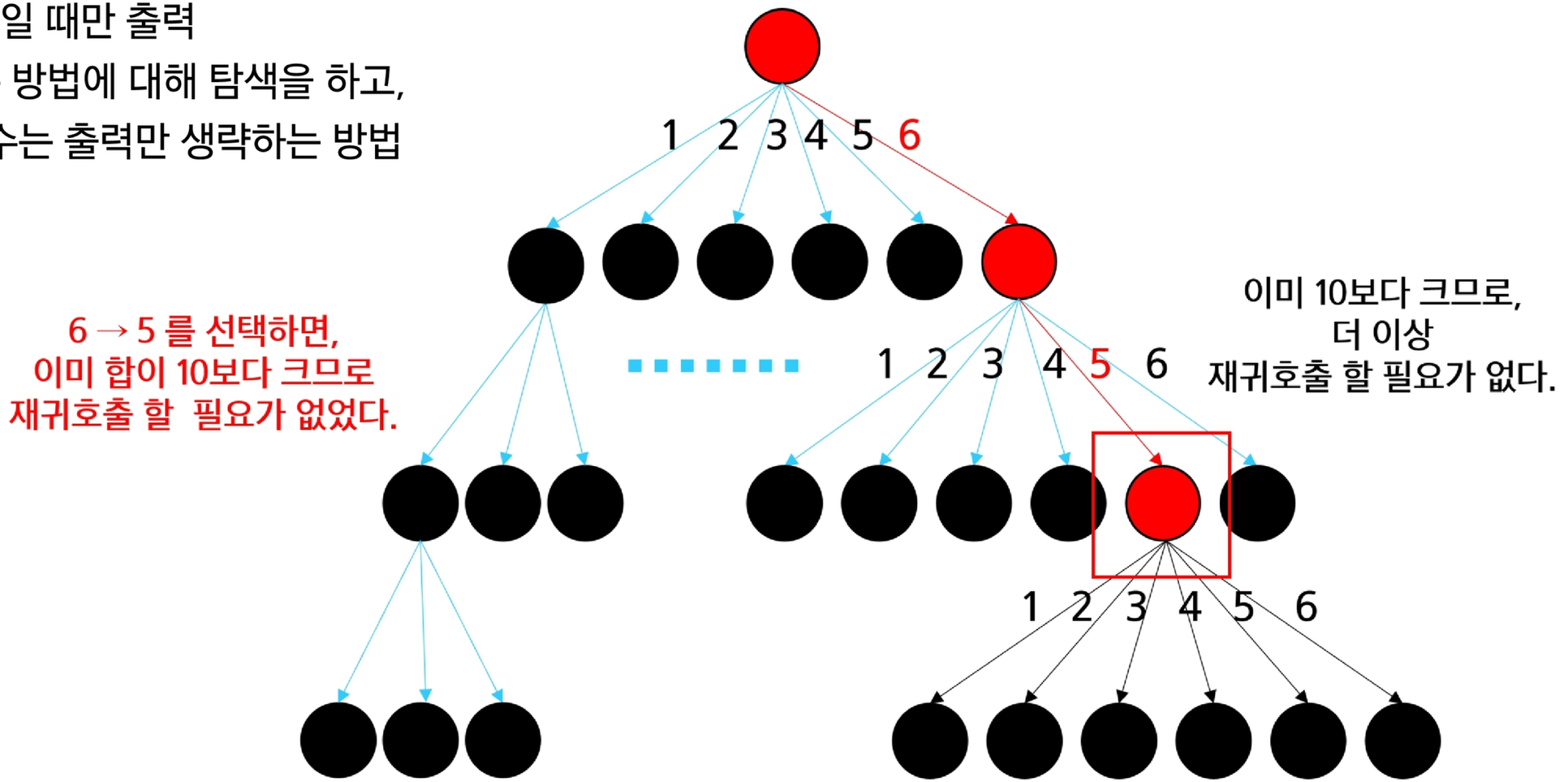
- Sum이 <= 10 이하 일 때만 출력
- 실제로는 모두 탐색하지만, 출력만 하지 않는 방법

```
path = []

def kfc(x, sum):
    if x == 3:
        if sum <= 10:
            print(f'{path} = {sum}')

        return

    for i in range(1, 7):
        path.append(i)
        kfc(x + 1, sum + i)
        path.pop()

kfc(x: 0, sum: 0)
```

완전탐색 문제 1. 주사위 눈의 합

- Sum이 <= 10 이하 일 때만 출력
- 위 소스코드는 모든 방법에 대해 탐색을 하고,

합이 10이 넘어가는 수는 출력만 생략하는 방법



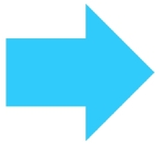
<그림29_알고리즘 응용_완전탐색_가지치기의 필요성>

완전탐색 문제 1. 주사위 눈의 합

- 가지치기: 답이 아닌 것에 대해 즉시 되돌아감

```
path = []

def kfc(x, sum):
    if x == 3:
        if sum <= 10:
            print(f'{path} = {sum}')

        return

    for i in range(1, 7):
        path.append(i)
        kfc(x + 1, sum + i)
        path.pop()

kfc(x: 0, sum: 0)
```

가지치기가 아니라 출력만 숨김 (느린 성능)



더욱 빠른 성능

```
path = []

def kfc(x, sum):
    if sum > 10:    # 가지치기
        return

    if x == 3:
        print(f'{path} = {sum}')

        return

    for i in range(1, 7):
        path.append(i)
        kfc(x + 1, sum + i)
        path.pop()

kfc(x: 0, sum: 0)
```

누적합이 10 넘어가는 순간 더 탐색할 필요 없음

완전탐색 문제 1. 주사위 눈의 합

- 합이 10 이하가 되는 경우의 수를 구하는 코드
  - 완성된 코드에서 시작합니다.
- 합이 10 이하일 때를 찾을 때 마다, cnt += 1을 수행
  - cnt값이 정답이 됩니다.
- 결과 108이 출력됩니다.

```
path = []
cnt = 0

def kfc(x, sum):
    global cnt
    if sum > 10:
        return

    if x == 3:
        # print(f'{path} = {sum}')
        cnt += 1
        return

    for i in range(1, 7):
        path.append(i)
        kfc(x + 1, sum + i)
        path.pop()

kfc(x: 0, sum: 0)
print(cnt)
```

완전탐색 문제 2. 연속 3장의 트럼프 카드

- A, J, Q, K 네 종류의 카드들이 충분히 있음
- 이 중, 5 장의 카드를 뽑아 나열

- A A A A A
- A A A A J
- …
- K K K K K

- 같은 종류의 카드가 세 장 연속으로 나오는 경우의 수는?

〈그림30_알고리즘 응용_완전탐색_4종류의 카드〉

완전탐색 문제 2. 연속 3장의 트럼프 카드

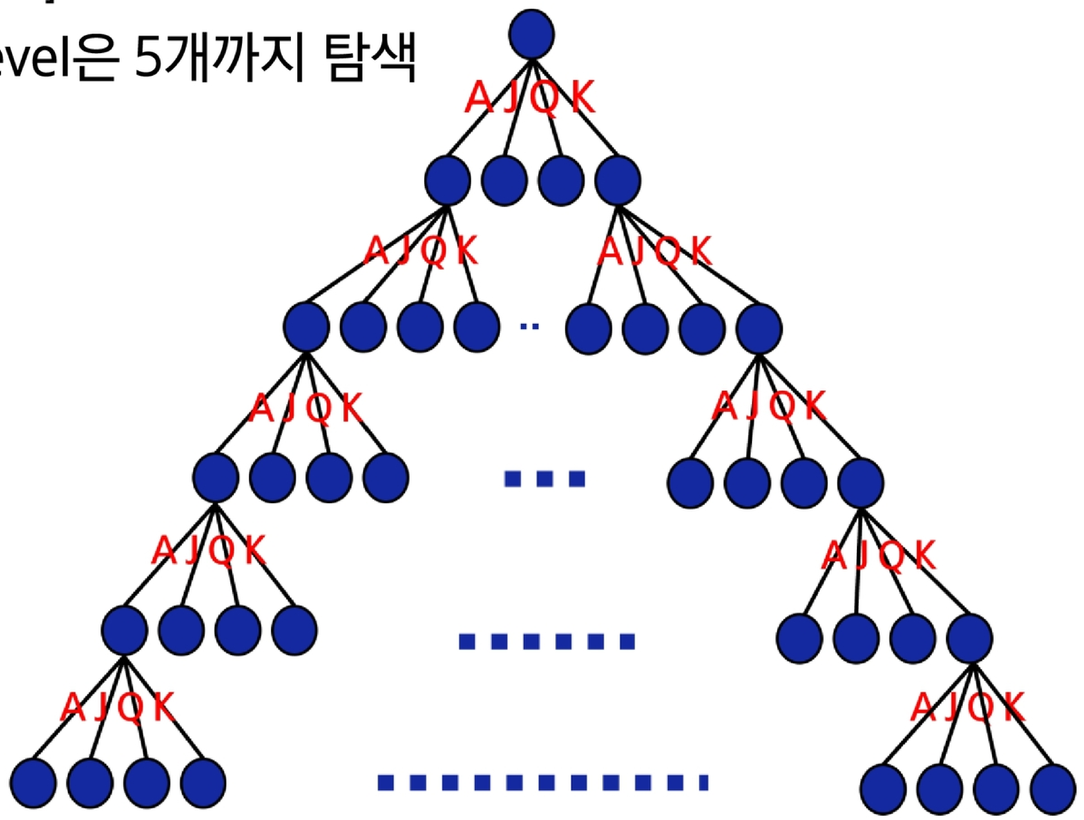
- 모든 경우에 대해 다 시도하기

① 순열 코드를 작성합니다.
- 먼저 [A, J, Q, K] 카드에 대해 [AAAAA ~ KKKKK] 까지 출력합니다.

③ 같은 종류의 카드가 연속으로 세 장이 나왔다면 counting 한다.

완전탐색 문제 2. 연속 3장의 트럼프 카드

- 순열 코드 작성하기
  - [AAAAA] ~ [KKKKK]
- Branch는 4개 Level은 5개까지 탐색



<그림31_알고리즘 응용_완전탐색_연속 3장의 트럼프 카드>

```
card = ['A', 'J', 'Q', 'K']
path = []

def permu(lev):
    if lev == 5:
        print(path)
        return

    for i in range(4):
        path.append(card[i])
        permu(lev + 1)
        path.pop()

permu(0)
```

완전탐색 문제 2. 연속 3장의 트럼프 카드

- 같은 카드가 연속 세장인지 검사하는 코드
- cont_three( )
  - 연속 세장이 있으면 True를 리턴
  - 아니면 False를 리턴

```
def cont_three():
    if path[0] == path[1] == path[2]: return True
    if path[1] == path[2] == path[3]: return True
    if path[2] == path[3] == path[4]: return True
    return False
```

완전탐색 문제 2. 연속 3장의 트럼프 카드

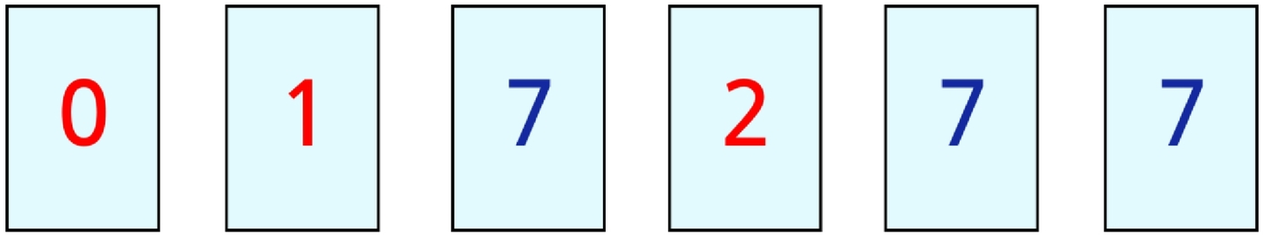
- 전체 코드

```
card = ['A', 'J', 'Q', 'K']
path = []
cnt = 0

def cont_three():
    if path[0] == path[1] == path[2]: return True
    if path[1] == path[2] == path[3]: return True
    if path[2] == path[3] == path[4]: return True
    return False
```

```
def permu(lev):
    global cnt
    if lev == 5:
        if cont_three():
            cnt += 1
        return

    for i in range(4):
        path.append(card[i])
        permu(lev + 1)
        path.pop()

permu(0)
print(cnt)
```

완전탐색 문제 3. Baby-gin

- 0이상 9이하의 번호가 적힌 6장의 카드



<그림32_알고리즘 응용_완전탐색_Baby-gin (ver. 완전탐색)>

- 3 장의 카드가 연속적인 번호를 갖는 경우 “run” 이라고 하고,
  3 장의 카드가 동일한 번호를 갖는 경우는 “triplet” 이라고 합니다.
- 그리고 6장의 카드가 run과 triplet로만 구성된 경우를 “baby-gin”으로 부릅니다.
  (위 예시에서는 0 1 2 와 7 7 7 카드가 있으므로 baby-gin 입니다.)

- 6장의 카드 번호를 입력 받고, 완전탐색으로 baby-gin 여부를 판단하는 프로그램을 작성하기

완전탐색 문제 3. Baby-gin

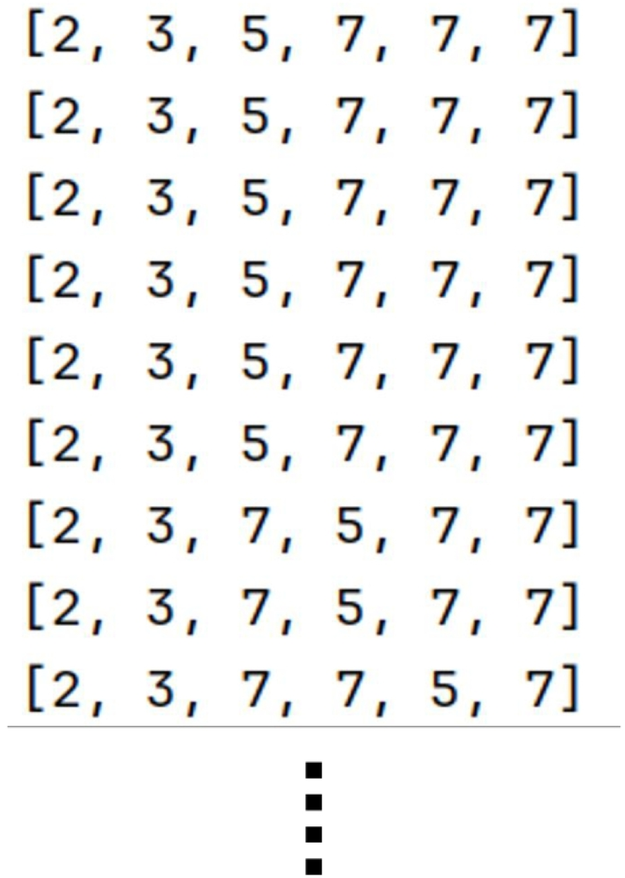
- 입력 예시
  - 667767 → 두 개의 triplet이므로 baby-gin 입니다. (666, 777)
  - 054060 → 한 개의 run과 한 개의 triplet이므로 역시 baby-gin 입니다. (456, 000)
  - 101123
    → 한 개의 triplet(111)이 존재하나, 나머지 023이 triplet, run이 아닙니다.
    → 한 개의 run(123)이 존재하나, 나머지 011이 triplet, run이 아닙니다.
    → 한 개의 run(012)이 존재하나, 나머지 113이 triplet, run이 아닙니다.
    → 따라서 baby-gin이 아닙니다.

- 완전탐색을 이용하여, baby-gin 여부를 검사하는 프로그램 작성하기

완전탐색 문제 3. Baby-gin

- 풀이 힌트 1. 고려할 수 있는 모든 경우의 수 생성하기
  - 6개의 카드로 만들 수 있는 모든 순서를 나열 = “순열" 코드를 작성합니다.

- 예) 입력으로 {2, 3, 5, 7, 7, 7}을 받았을 경우, 그림과 같이 순열을 생성

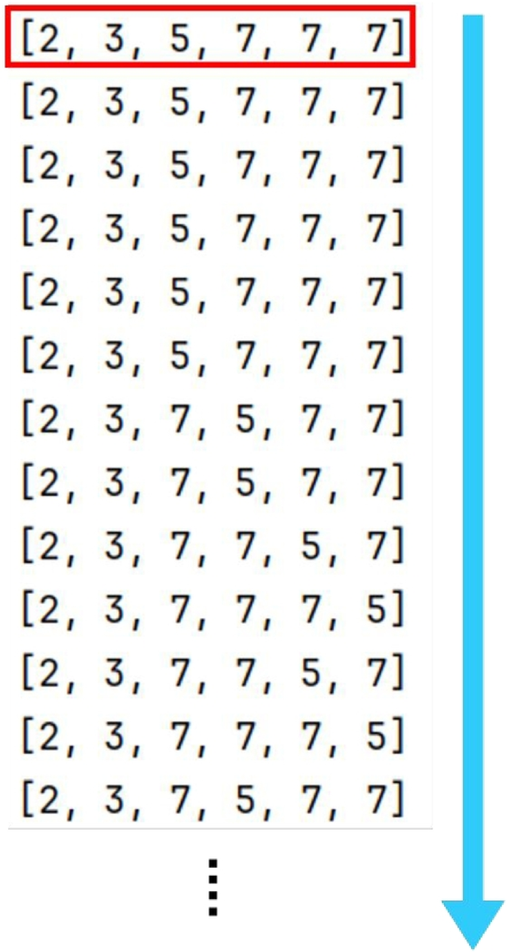


<그림33_알고리즘 응용_완전탐색_풀이 힌트 1>

완전탐색 문제 3. Baby-gin

- 풀이 힌트 2. 검사하는 함수를 제작하여 하나씩 테스트
- is_baby_gin( ) 함수를 제작
  - 앞 3 자리가 run 또는 triplet 이면서
    뒤 3 자리가 run 또는 triplet 이면 True를 리턴합니다.
  - 그렇지 않으면 False를 리턴한다.
- 많은 순열 중, 하나라도 True라면 Baby-gin

<그림35_알고리즘 응용_완전탐색_Baby-gin 검사>



<그림34_알고리즘 응용_완전탐색_풀이 힌트 2>

연습 문제

연습문제 1 : Baby-gin

- 6자리 숫자를 입력 받은 후, 완전탐색을 적용하여 Baby-gin 여부를 검사하는 프로그램을 작성 (Yes / No 를 출력)

- 입력 / 출력 예시
  - 1 2 4 7 8 3 → No
  - 6 6 7 7 6 7 → Yes
  - 0 5 4 0 6 0 → Yes
  - 1 0 1 1 2 3 → No

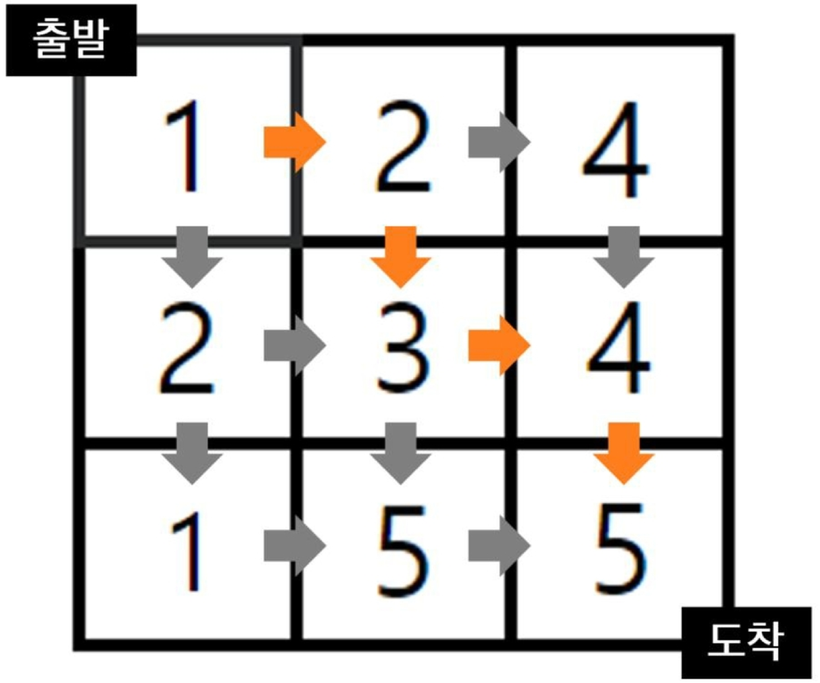
실습 문제

실습 문제1 - 5188. 최소합

(0, 0) 에서 출발하여,
오른쪽➡과 아래⬇로만 이동할 수 있다.

숫자 판을 밟을 때 마다 에너지가 소모된다.

(N - 1, N - 1) 에 도착 했을 때
최소 에너지 소모량의 합계를 구하여라



위 예시 경우,
1 + 2 + 3 + 4 + 5
= 15가 최소로 정답이 된다.

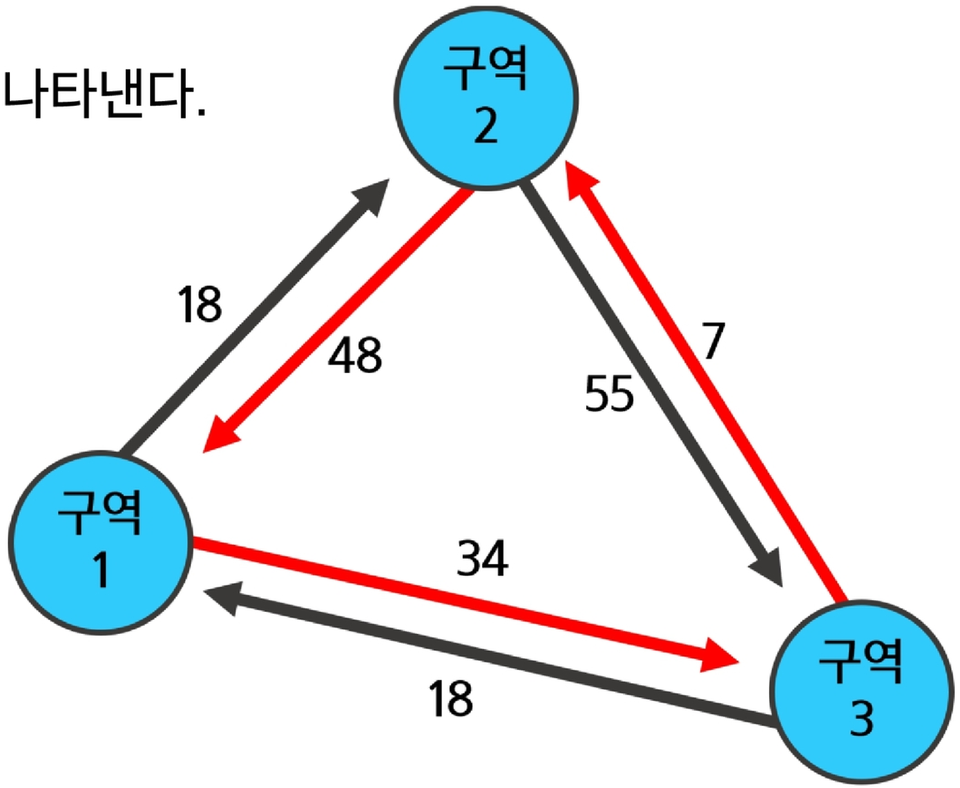
<그림36_알고리즘 응용_완전탐색_5188. 최소합>

실습 문제2 - 5189. 전자카트

1 ~ N 번 구역이 있다.
각 구역에서 다른 구역으로 카트를 타고 이동한다.
이동할 때마다 배터리가 소모되며, 아래 그림은 소모되는 비용을 나타낸다.

1번 구역에서 출발해서 모든 구역을 다 한번씩 들린 후,
다시 1번으로 돌아오려고 한다.

최소 배터리 소모량을 출력하라.



34 + 7 + 48 = 89가
최소 배터리 소모량이 된다.

<그림37_알고리즘 응용_완전탐색_5189. 전자카트>

실습

완전탐색

- 5188. 최소합

순열

- 5189. 전자카트

재귀

- 2105. [모의 SW 역량테스트] 디저트 카페

추가 연습

- 1247. [S/W 문제해결 응용] 3일차 - 최적 경로
- 1244. 최대상금

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 질문에 답하시오. (단답형)

어떤 조건을 만족하는 모든 부분집합을 찾을 때 사용하는 탐색 기법은?

2 완전탐색에서 재귀 호출을 사용하는 주된 이유는?

a) 실행 속도를 빠르게 하기 위해
b) 코드 복잡도를 높이기 위해
c) 반복문을 대체하여 선택 흐름을 자연스럽게 표현하기 위해
d) 함수 호출 횟수를 줄이기 위해

3 완전탐색으로 해결할 수 있는 경우라도, 현실적으로 사용하지 않는 이유는?

a) 정답이 없기 때문
b) 코드 작성이 불가능하기 때문
c) 계산량이 너무 많아 시간이 오래 걸리기 때문
d) 논리적으로 오류가 있기 때문

4 다음 질문에 답하시오. (단답형)

재귀 함수를 사용하는 완전탐색에서는 매 단계마다 어떤 방식으로 선택을 이어가나요?

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 완전탐색의 시간 복잡도가 급격히 증가하는 주된 이유는?

a) 조건 비교가 어려워서
b) 반복문 대신 재귀를 사용해서
c) 모든 경우를 탐색하기 때문
d) 배열 접근이 느려서

6 다음 중 재귀를 사용한 완전탐색 코드의 종료 조건으로 가장 적절한 것은?

a) 함수가 세 번 호출되었을 때
b) 현재 선택한 조합이 중복되었을 때
c) 스택이 비었을 때
d) 선택한 원소의 수가 목표 개수에 도달했을 때

7 다음 질문에 답하시오. (단답형)

반복문으로 순열을 구현하기 어려운 이유는 무엇인가요?

8 완전탐색으로 "부분집합의 합이 특정 값이 되는 경우"를 찾는 데 가장 적절한 기법은?

a) 해시 테이블
b) 재귀 호출 기반 부분집합 생성
c) 이진 탐색
d) 정렬

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 가지치기를 적용한 완전탐색 코드의 특징은?

a) 탐색 결과가 정확하지 않을 수 있다
b) 탐색 속도는 느리지만 모든 경우를 확인한다
c) 조건에 따라 탐색 깊이를 늘린다
d) 조건에 따라 탐색을 중단하여 성능을 높인다

10 다음 중 완전탐색 기법이 적합한 문제 유형은 무엇인가요?

a) 정렬된 리스트에서 특정 값을 찾는 문제
b) 모든 조합 중에서 조건을 만족하는 조합을 찾는 문제
c) 트리의 최소 높이를 구하는 문제
d) 그래프에서 최단 경로를 찾는 문제

11 다음 중 완전탐색으로 해결하기 어려운 문제는?

a) 3자리 비밀번호의 모든 경우 탐색
b) 10명의 사람 중 3명을 뽑는 경우
c) 50명 중 25명을 뽑는 최적 조합 찾기
d) 두 개 숫자의 곱을 계산하는 문제

12 "가지치기"에 대한 설명으로 가장 적절한 것은?

a) 모든 경우의 수를 끝까지 탐색하는 방법
b) 조건에 맞는 결과만 출력하는 방법
c) 유망하지 않은 경로를 조기에 탐색 종료하는 전략
d) 반복문 없이 탐색을 수행하는 전략

확인문제

정답 및 해설

1.완전탐색 (또는 부분집합 생성 재귀) 2. c) 반복문을 대체하여 선택 흐름을 자연스럽게 표현하기 위해 3. c) 계산량이 너무 많아 시간이 오래 걸리기 때문 4. 현재 단계 선택 후, 다음 단계로 재귀 호출 5. c) 모든 경우를 탐색하기 때문 6. d) 선택한 원소의 수가 목표 개수에 도달했을 때 7. 선택한 요소를 제외한 나머지를 재귀적으로 탐색해야 하기 때문 8. b) 재귀 호출 기반 부분집합 생성 9. d) 조건에 따라 탐색을 중단하여 성능을 높인다 10. b) 가능한 모든 조합 중에서 조건을 만족하는 조합을 찾는 문제 11. c) 50명 중 25명을 뽑는 최적 조합 찾기 12. c) 유망하지 않은 경로를 조기에 탐색 종료하는 전략

1. 모든 원소를 포함할지 말지를 결정하면서 모든 부분집합을 생성하는 것이 완전탐색입니다.

2. 재귀 호출은 선택과 조건 검사 흐름을 단계별로 분기하면서 자연스럽게 표현할 수 있어 완전탐색에 적합합니다.

3. 완전탐색은 계산량이 많아질수록 실행 시간이 비현실적으로 길어질 수 있다는 단점이 있습니다.

4. 재귀 호출을 통해 선택을 누적하고, 다음 단계의 선택으로 이어가는 흐름을 표현합니다

5. 완전탐색은 가능한 경우의 수를 하나도 빠짐없이 확인하므로, 경우의 수가 많아질수록 시간 복잡도가 급격히 증가합니다.

6. 재귀 기반 완전탐색은 일반적으로 목표 깊이 또는 선택 개수 도달 시 종료 조건을 설정하여 탐색을 마칩니다.

7. 순열은 요소의 순서까지 고려하므로, 매 단계마다 선택한 값을 제외하고 다시 탐색해야 합니다. 반복문만으로 구현하기 어렵습니다.

8. 부분집합의 합 문제는 모든 부분집합을 만들어 합을 계산해야 하므로, 재귀 호출 방식이 가장 자연스럽고 효과적입니다.

9. 가지치기는 탐색을 일찍 끝내 불필요한 재귀 호출을 줄이는 데 목적이 있습니다.

10. 완전탐색은 경우의 수를 전부 살펴봐야 하므로, 조건을 만족하는 조합을 찾는 데 적합합니다.

11. 경우의 수가 너무 많아져 현실적으로 계산 시간이 폭발적으로 증가하므로 완전탐색이 부적절합니다.

12. 가지치기는 탐색 도중 더 이상 의미 없는 경우를 제거하여 탐색 효율을 높이는 기법입니다.

활동정리

오늘 공부한 내용 요약 및 정리

- **완전 탐색이란?**

- 가능한 모든 경우를 만들어 보고 그 중에서 조건을 만족하는 정답을 찾는 기본적인 문제 해결 전략
- 가장 단순하지만, 정답이 보장되고 구현이 직관적인 장점이 있음
- 경우의 수가 많아질수록 시간이 오래 걸릴 수 있어, 효율성보다는 정확성 중심의 방식

- **완전탐색 구현 방식**

- **중첩 반복문**을 이용해 직접 모든 조합을 만들 수 있음
- **재귀 호출**을 사용하여 반복 구조를 더 유연하고 간결하게 표현 가능

예) 중첩 반복문으로 두 주사위 눈의 합 구하기, run(lev) 형식의 재귀 함수로 모든 조합 생성

활동정리

오늘 공부한 내용 요약 및 정리

• 재귀 호출 구조와 흐름

- 현재 단계에서 하나의 선택을 한 후, 다음 단계로 넘어가기 위해 자기 자신을 호출함
- 각 호출에는 기저 조건이 존재하여 더 이상 진행하지 않고 종료되는 조건을 명확히 설정해야 함
- 깊이가 조건을 만족하면 return으로 종료함

• 순열, 중복 순열, 부분집합 생성

- **순열**: 순서를 고려하여 중복 없이 선택
- **중복 순열**: 순서를 고려하되 같은 값을 여러 번 선택 가능
- **부분집합**: 각 원소를 선택할지 말지를 이진 분기로 탐색
- 각각 재귀를 통해 쉽게 구현 가능하며, 호출 흐름이 자연스럽게 트리 형태로 확장

활동정리

오늘 공부한 내용 요약 및 정리

• 조건에 따른 탐색 제한 (가지치기)

- 모든 경우를 다 볼 필요 없이, 중간에 조건을 만족하지 않으면 그 경로는 탐색하지 않음
- 예: 주사위 눈의 합이 특정 값을 초과한 경우 return으로 조기 종료
- 효율적인 완전탐색을 위해 반드시 필요

• 재귀 호출의 동작 원리 시각화

- 호출과 복귀가 쌓이고 돌아가는 구조는 **스택 구조(LIFO)**로 이해 가능
- 각 호출이 완료되면 이전 호출로 되돌아가며, 전체 탐색이 마무리됨
- 이를 통해 함수의 흐름과 선택 분기 과정을 체계적으로 분석 가능

활동정리

핵심키워드

개념	설명	예시
완전 탐색	가능한 모든 경우의 수를 생성하고 조건을 검사하여 정답을 찾는 방식	두 개의 주사위를 던져 나올 수 있는 모든 조합을 출력하고, 이 중 눈의 합이 10 이상인 경우만 골라내는 방식.
중첩 반복문	반복문 안에 반복문을 중첩하여 여러 경우를 직접 구현	2중 for문으로 1~3까지 모든 쌍 출력 (1 1 ~ 3 3)
재귀 호출	함수가 자기 자신을 호출하여 중첩 반복 구조를 표현	3장의 트럼프 카드를 선택하는 모든 순서를 출력하기 위해 run(lev) 함수를 사용. 각 단계에서 카드 하나를 선택하고, 다음 레벨로 재귀 호출하여 모든 순열을 출력
순열	중복 없이 n개 중 r개를 골라 순서대로 늘어놓은 것	카드 [0, 1, 2] 중 2장을 고르는 순열 → 0 1,0 2,1 0,1 2,…
중복 순열	중복을 허용하고 순서를 고려하여 n개 중 r개를 고르는 경우의 수	주사위 2개 던지기 → [1 1], [1 2],…,[6 6] 등
기저 조건	재귀 함수가 더 이상 자기 자신을 호출하지 않고 종료하는 조건	`if lev == N: return` → 완전 탐색 종료 조건 설정
가지치기 (조건 탈출)	조건을 만족하지 않으면 더 이상 재귀를 진행하지 않고 종료	주사위 눈 합이 10 초과면 더 탐색하지 않고 return

<표1_알고리즘 응용_완전탐색_핵심 키워드>

활동정리

조건에 맞는 코디를 골라 봅시다!

옷장에 상의 4벌(A, B, C, D)과 하의 2벌(a, b)이 있습니다. 각각 한 벌씩 골라 입으려면 어떤 코디가 가능할까요? 단, A 상의와 b 하의는 어울리지 않기 때문에 함께 입을 수 없습니다.

→ 가능한 모든 조합을 하나씩 만들어 보고, 조건에 맞는 경우만 선택하는 방식을 [ ]이라고 합니다.

→ 한 가지 선택을 하고, 남은 경우를 이어서 고르는 흐름은 [ ] 구조로 표현할 수 있습니다.

상의 4벌(A, B, C, D), 하의 2벌(a, b)로 만들 수 있는 모든 조합은 4 × 2 = 8가지입니다.
단, A 상의와 b 하의는 함께 입을 수 없으므로 해당 조합을 제외하면 총 7가지 유효한 코디가 됩니다.

→ 이처럼 모든 가능한 조합을 생성한 뒤, 조건을 검사해 정답을 골라내는 방식을 완전탐색이라고 합니다.
→ 문제 해결 과정에서 하나의 선택을 하고, 그 다음 선택으로 이어가는 흐름은
코드에서 함수가 자기 자신을 다시 호출하는 재귀 호출 구조로 표현할 수 있습니다.

활동정리

조건에 맞는 코디를 골라 봅시다!

옷장에 상의 4벌(A, B, C, D)과 하의 2벌(a, b)이 있습니다. 각각 한 벌씩 골라 입으려면 어떤 코디가 가능할까요? 단, A 상의와 b 하의는 어울리지 않기 때문에 함께 입을 수 없습니다.

→ 가능한 모든 조합을 하나씩 만들어 보고, 조건에 맞는 경우만 선택하는 방식을 완전탐색 이라고 합니다.

→ 한 가지 선택을 하고, 남은 경우를 이어서 고르는 흐름은 재귀 호출 구조로 표현할 수 있습니다.

상의 4벌(A, B, C, D), 하의 2벌(a, b)로 만들 수 있는 모든 조합은 4 × 2 = 8가지입니다.
단, A 상의와 b 하의는 함께 입을 수 없으므로 해당 조합을 제외하면 총 7가지 유효한 코디가 됩니다.

→ 이처럼 모든 가능한 조합을 생성한 뒤, 조건을 검사해 정답을 골라내는 방식을 완전탐색이라고 합니다.
→ 문제 해결 과정에서 하나의 선택을 하고, 그 다음 선택으로 이어가는 흐름은
코드에서 함수가 자기 자신을 다시 호출하는 재귀 호출 구조로 표현할 수 있습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다